POLITECNICO DI MILANO





Machine Learning

Andrea Mor
andrea.mor@polimi.it

Machine learning and artificial intelligence

Rate your ML knowledge

pollev.com/andreamor103

Get the slides

github.com/mrondr/MDACS26

Machine learning and artificial intelligence

Artificial Intelligence

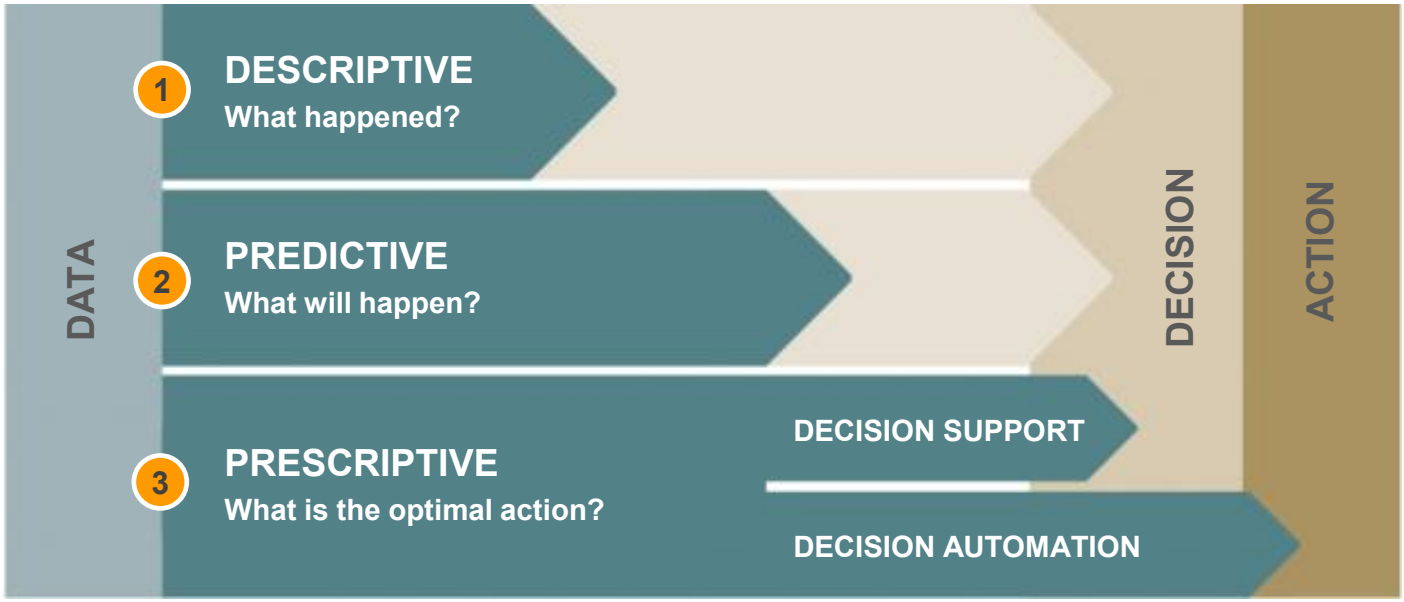
Is the study of "intelligent agents“, often represented by machines, which mimic cognitive human functions (perceive the environment and take actions) to obtain a specific goal.

Machine Learning

Subset of AI focused on algorithms that enable computers to learn from data and improve over time without explicit programming.

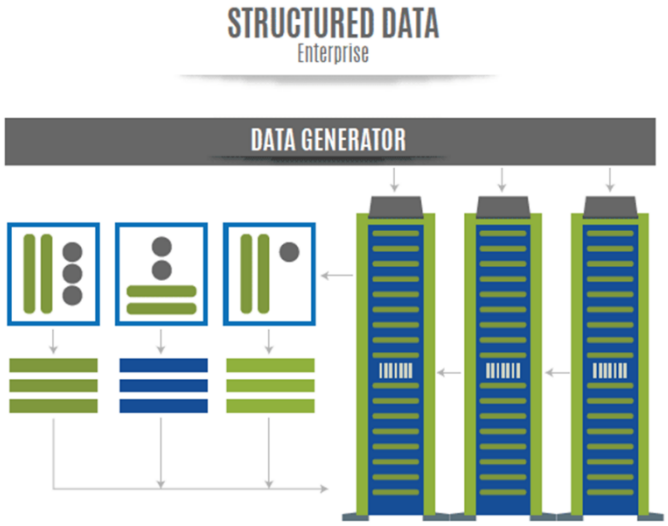
From descriptive to prescriptive analytics

Scheme adapted from the original work by Gartner

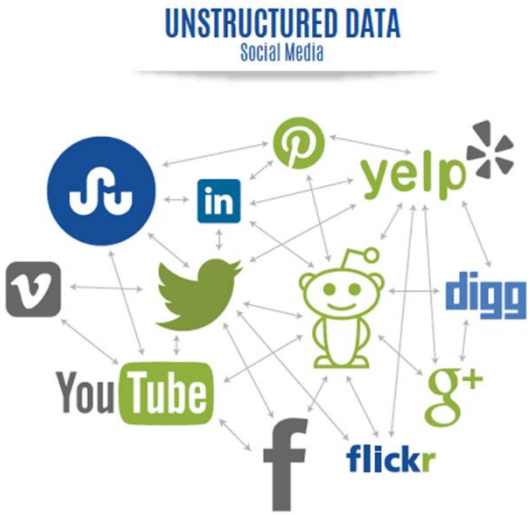


1. Analyze historical data to look for the reasons behind a given phenomenon.
2. Use algorithms to discover patterns in historical data and predict the future outcome of an event. Patterns should be novel, potentially useful, understandable.
3. Suggest the best actions to take advantage of predictions.

Data sources

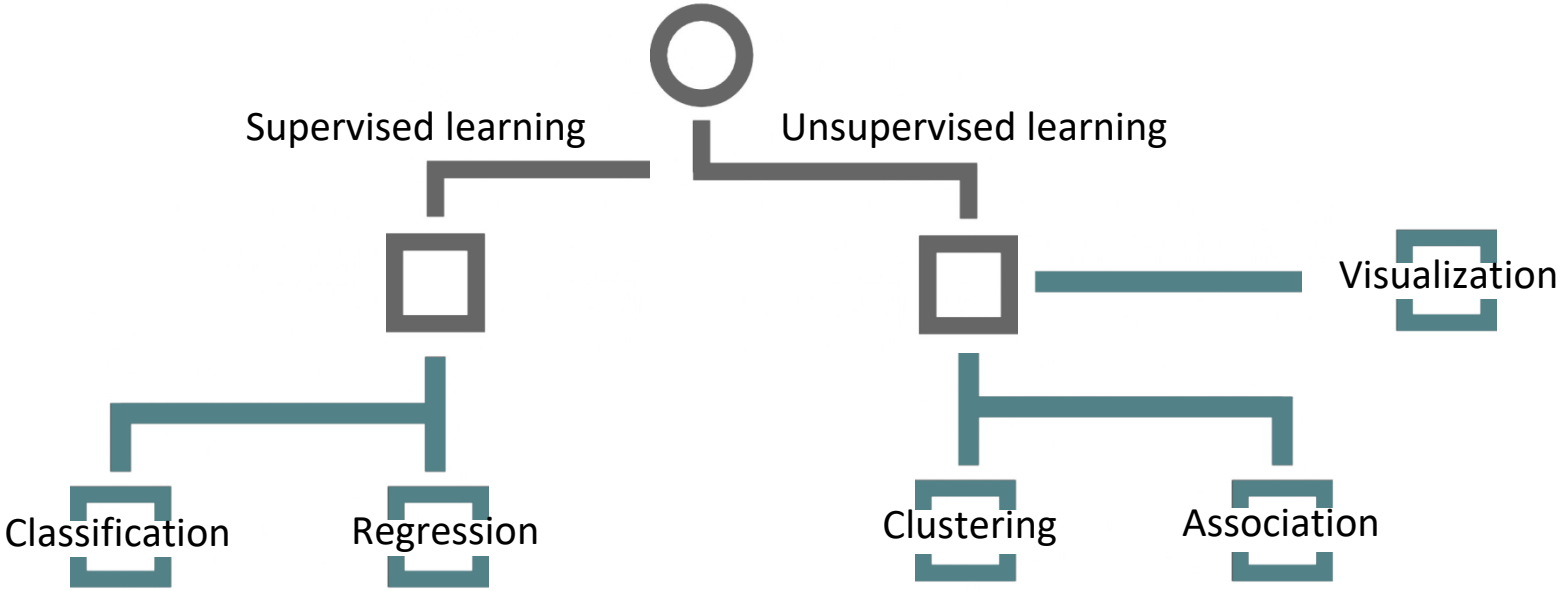


- ✓ Traffic data
- ✓ Commercial transactions
- ✓ Sensor data (GPS, medical devices, smart meters, etc.)
- ✓ Web server logs
- ✓ Financial data
- ✓ …

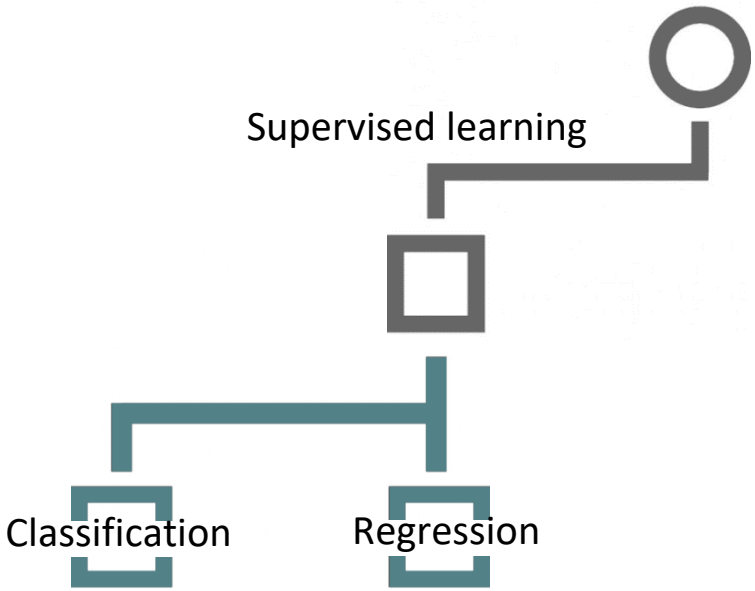


- ✓ Social media posts
- ✓ Digital images and videos
- ✓ Audio files
- ✓ …

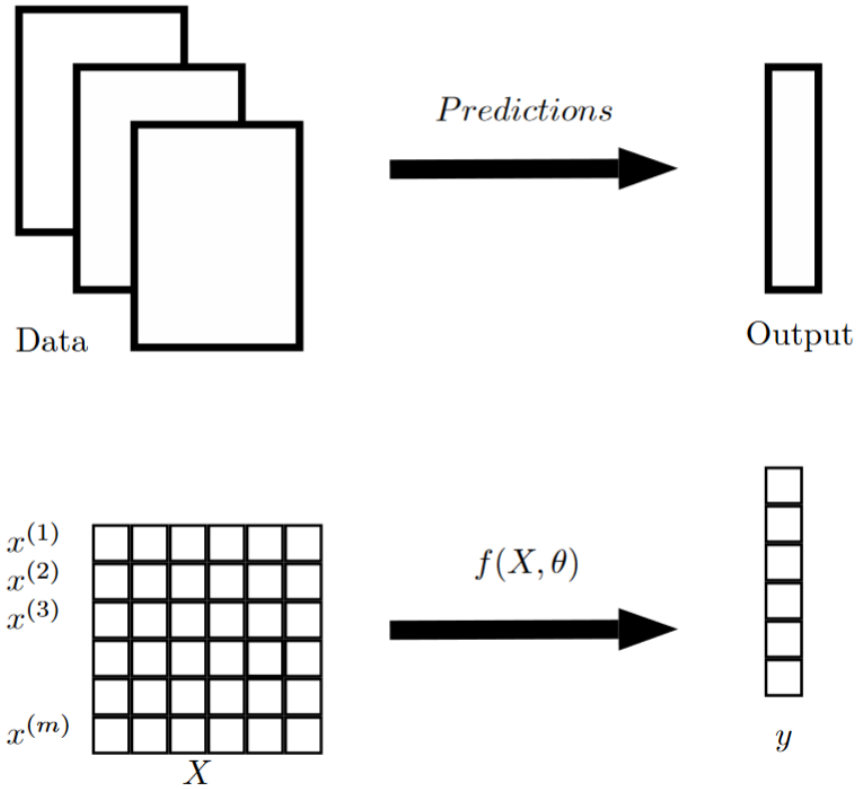
Machine learning tasks



Supervised Learning



Supervised learning



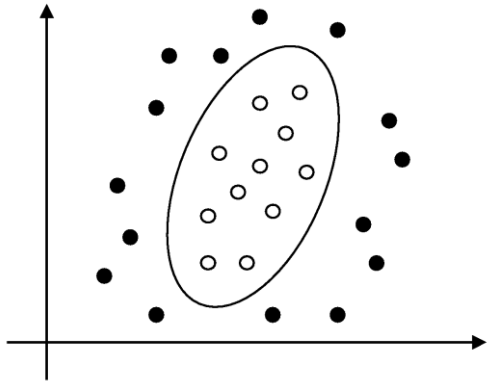
Supervised learning:

Given a set D of data points $D=\{\mathbf{X}_i, Y_i\}$ ($i \in M$), infer a function $Y_i = f(\mathbf{X}_i)$ mapping the inputs to the output.

CLASSIFICATION

Divide a dataset of examples into classes according to their characteristics with the aim of predicting the class of novel future examples.

Example: Divide patients into two classes based on the presence/ absence of a disease and identify the individuals who are most at risk for prevention purposes.

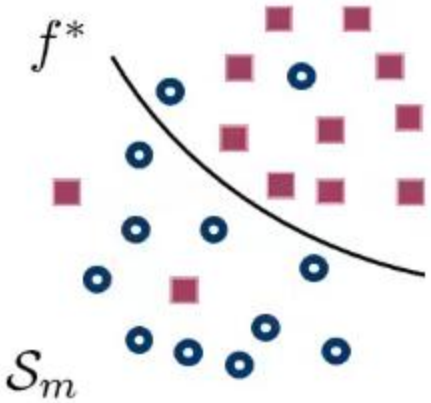


Classification

$\mathcal{S}_m = \{(\mathbf{x}_i, y_i),\ i \in \mathcal{M}\}$: **training set, where** $\mathbf{x}_i \in \Re^n$ **and** $y_i \in \mathcal{D}$

$\mathcal{H}$ **denotes a set of functions** $f(\mathbf{x}) : \Re^n \mapsto \mathcal{D}$

Classification problem: define a hypotheses space $\mathcal{H}$ **and a function** $f^* \in \mathcal{H}$ **which optimally describes the relationship between** $\mathbf{x}_i$ **and** y_i



$\Rightarrow \quad f^* \in \mathcal{H} \quad \Rightarrow \quad \overline{\mathbf{x}} \quad \Rightarrow \quad f(\overline{\mathbf{x}}) = -1$

learning prediction

new patient

$\Downarrow$

healthy !

healthy patient (-1)

ill patient (+1)

REGRESSION

Predict the value of a numeric variable for each example in a dataset based on the values of other characteristics.

Example: Predict the life expectation of a individual based on different characteristics (socio-demographic factors, life-style, profession, clinical tests,…).

$$Y = f(X1, X2, \ldots, Xn)$$

Dataset example

door
machine learning group
POLITECNICO DI MILANO

The dataset contains information about some failures occurred on a machine while processing some products.

Product ID	UDI	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Failure Type	Failure
L47183	4	L	298.2	308.6	1433	39.5	7	No Failure	0
M14865	6	M	298.1	308.6	1425	41.9	11	No Failure	0
L47186	7	L	298.1	308.6	1558	42.4	14	No Failure	0
L47213	34	L	298.9	309.3	1665	32.5	93	No Failure	0
L47219	40	L	298.8	309.1	1350	52.5	111	No Failure	0
M14902	43	M	298.8	309.1	1368	49.1	117	No Failure	0
L47230	51	L	298.9	309.1	2861	4.6	143	Power Failure	1
L47234	55	L	298.7	309.0	1691	30.1	154	No Failure	0
M14923	64	M	298.9	309.0	1812	25.9	174	No Failure	0
L47249	70	L	298.9	309.0	1410	65.7	191	Power Failure	1
L47257	78	L	298.8	308.9	1455	41.3	208	Tool Wear Failure	1
M14957	98	M	298.9	308.9	1750	29.9	50	No Failure	0
L47278	99	L	298.9	308.8	1529	32.7	53	No Failure	0
L47296	117	L	298.9	308.7	1564	34.1	99	No Failure	0
H29557	144	H	298.6	308.4	1407	50.5	164	No Failure	0
M15008	149	M	298.3	308.3	1379	48.0	181	No Failure	0
M15018	159	M	298.4	308.2	1499	40.0	211	No Failure	0
L47340	161	L	298.4	308.2	1282	60.7	216	Overstrain Failure	1

Quality of the product

Force that tends to cause the rotation around an axis

Time before observing the gradual failure of the cutting tools

Another dataset

door
machine learning group
POLITECNICO DI MILANO

The data set contains the log of bike trips as recorder by the software operating a bike sharing system

lngPKID	lngFKIDBike	lngFKIDBadge	dtmDateTimePickup	dtmDateTimeReturn	lngFKIDStationPickup	intStandPickup	lngFKIDStationReturn	intStandReturn
5074	213	4704	2011-08-28 00:04:43	2011-08-28 18:12:36	22	3	20	7
5106	213	3298	2011-08-28 11:19:04	2011-08-28 11:43:07	37	4	35	13
5114	213	5688	2011-08-28 12:19:17	2011-08-28 12:34:20	35	13	20	7
5151	213	4	2011-08-28 18:19:26	2011-08-28 18:57:44	20	7	33	4
5166	213	6059	2011-08-28 19:49:43	2011-08-28 19:56:53	33	4	37	7
5177	213	2534	2011-08-28 21:00:57	2011-08-28 21:03:42	37	7	31	4
5187	213	5956	2011-08-28 22:00:58	2011-08-28 22:18:24	31	4	36	5
5198	213	5956	2011-08-28 23:17:54	2011-08-28 23:22:04	36	5	31	1
5203	213	3861	2011-08-29 06:48:53	2011-08-29 07:09:19	31	1	6	8
5560	213	3053	2011-08-29 17:29:12	2011-08-29 17:29:40	6	8	6	8
5576	225	5949	2011-08-29 17:44:01	2011-08-29 21:01:36	25	16	25	12
5583	225	5949	2011-08-29 17:53:58	2011-08-29 20:01:03	25	27	25	12
5584	225	5949	2011-08-29 17:54:41	2011-08-29 17:55:18	25	28	25	12
5717	213	3	2011-08-30 07:33:10	2011-08-30 07:57:57	6	8	25	29
5780	225	2444	2011-08-30 08:43:43	2011-08-30 22:52:00	25	12	30	4
5807	225	5778	2011-08-30 09:17:12	2011-08-30 09:21:32	20	8	19	1
5814	213	5516	2011-08-30 09:27:58	2011-08-30 09:33:43	22	13	38	8
5836	213	5697	2011-08-30 10:06:36	2011-08-30 10:20:55	38	8	6	7
5908	213	5474	2011-08-30 11:59:05	2011-08-30 12:19:12	6	7	38	1
5916	225	3644	2011-08-30 12:06:28	2011-08-30 20:06:13	19	1	37	9
5948	225	3644	2011-08-30 12:52:35	2011-08-30 18:25:25	19	1	37	9

What is the target in this case?

pollev.com/andreamor103

Can you think of some more applications for these methodologies?

pollev.com/andreamor103

Supervised learning applications

door
machine learning group
POLITECNICO DI MILANO

Retail

Find segments of customers who are likely to respond to a marketing campaign.

Forecast the demand for a given product.

Find which products are often bought together.

Banking

Find segments of individuals at high risk of insolvency (granting a loan).

Identify anomalous transactions.

Insurance

Identify fraudolent behaviours.

Find segments of customers who are likely to churn.

Healthcare

Find groups of patients who are more likely to be re-hospitalized.

Find genes which are highly informative against the presence of a given pathology.

Manufacturing

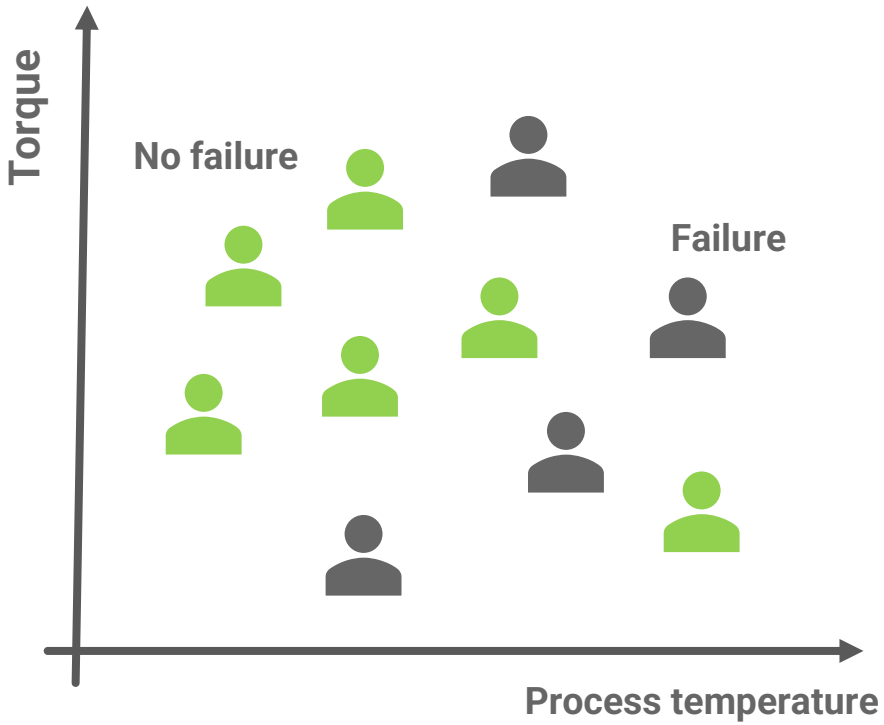
Build models for early failure detection or quality control.

...cross-domain analysis

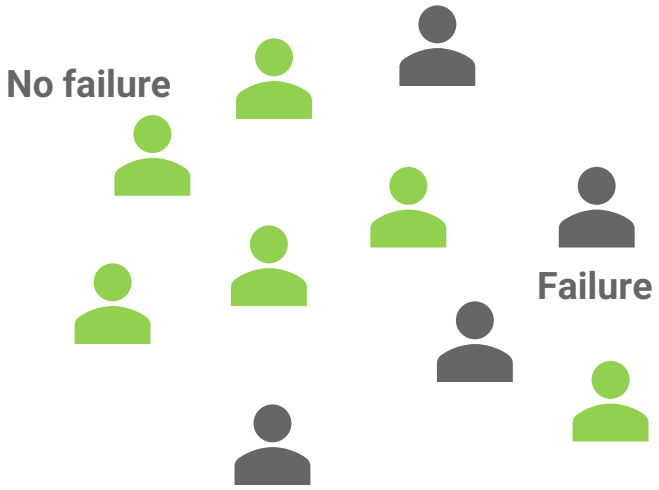
Image and video recognition, text classification (sentiment analysis), signal recognition, etc...

Our examples

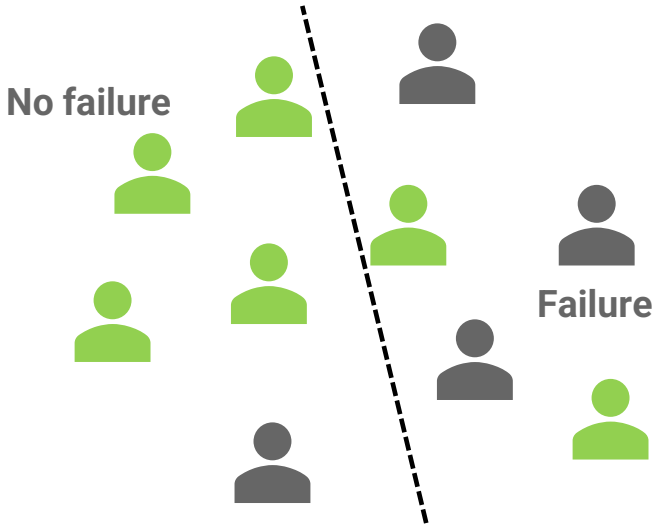
Examples are points in the n-dimensional feature space.



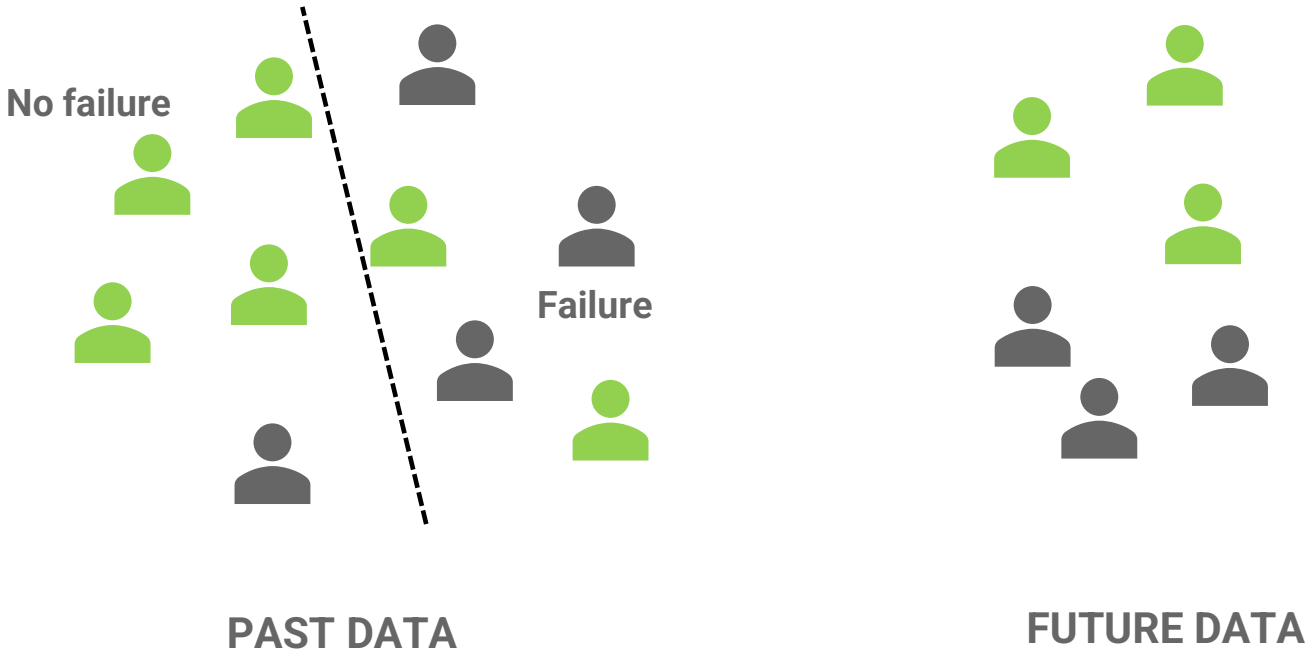
Classification: how to build the model



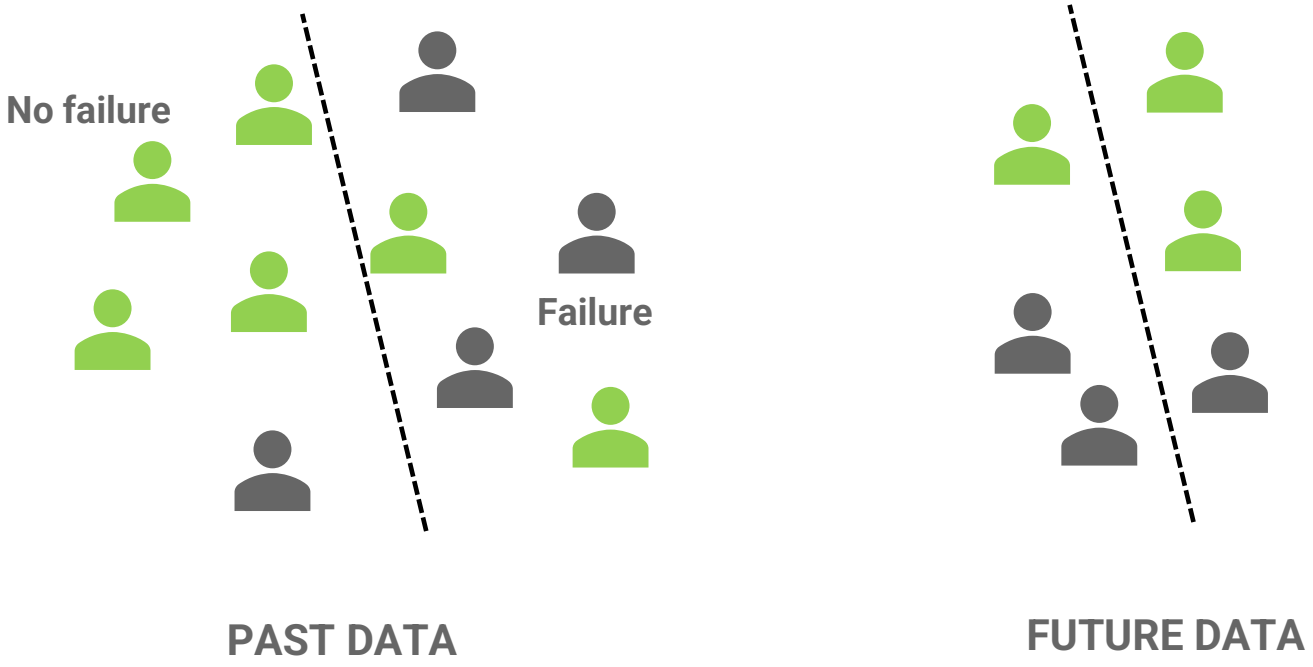
Classification: how to build the model



Classification: how to build the model

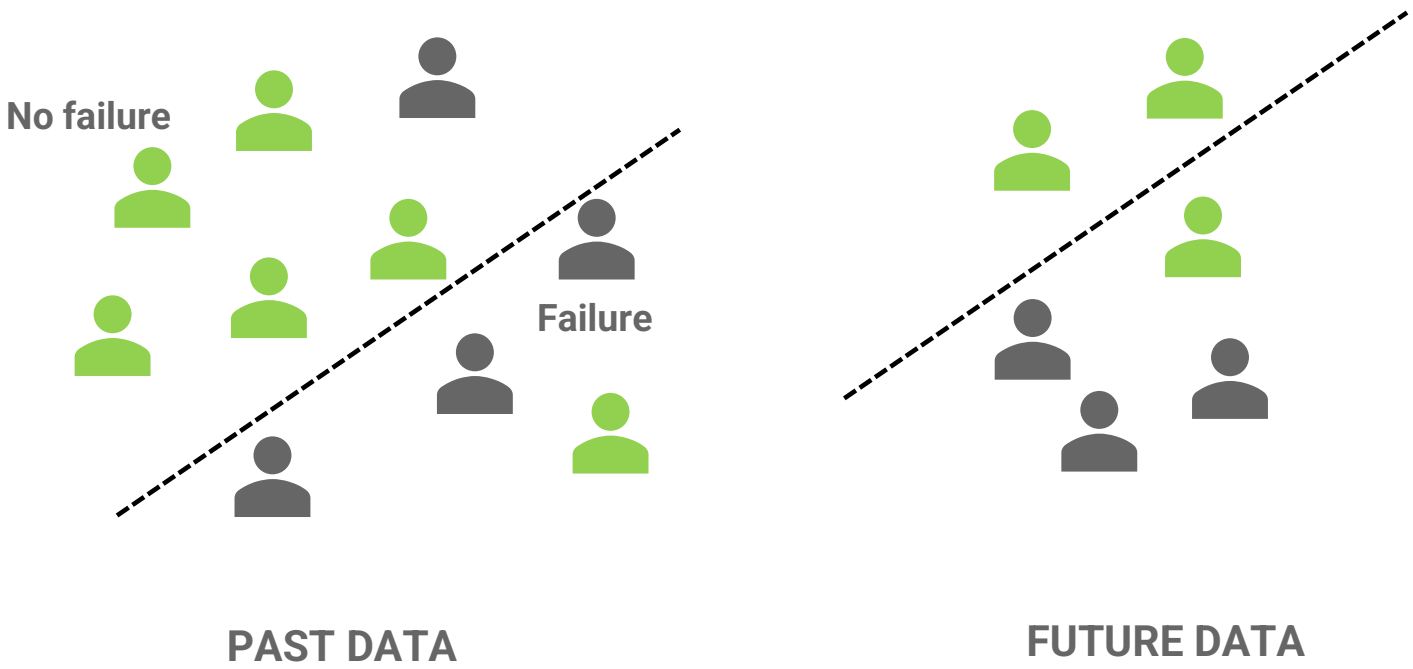


Classification: how to build the model

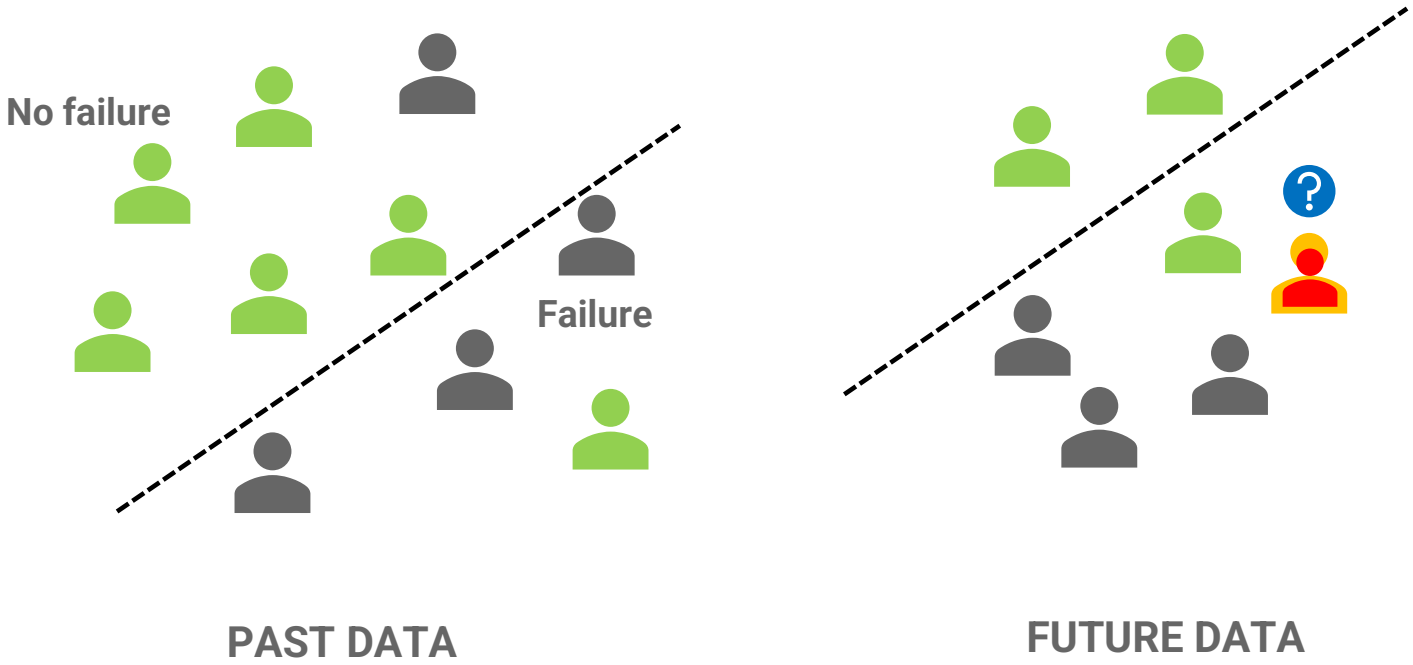


Classification: how to build the model

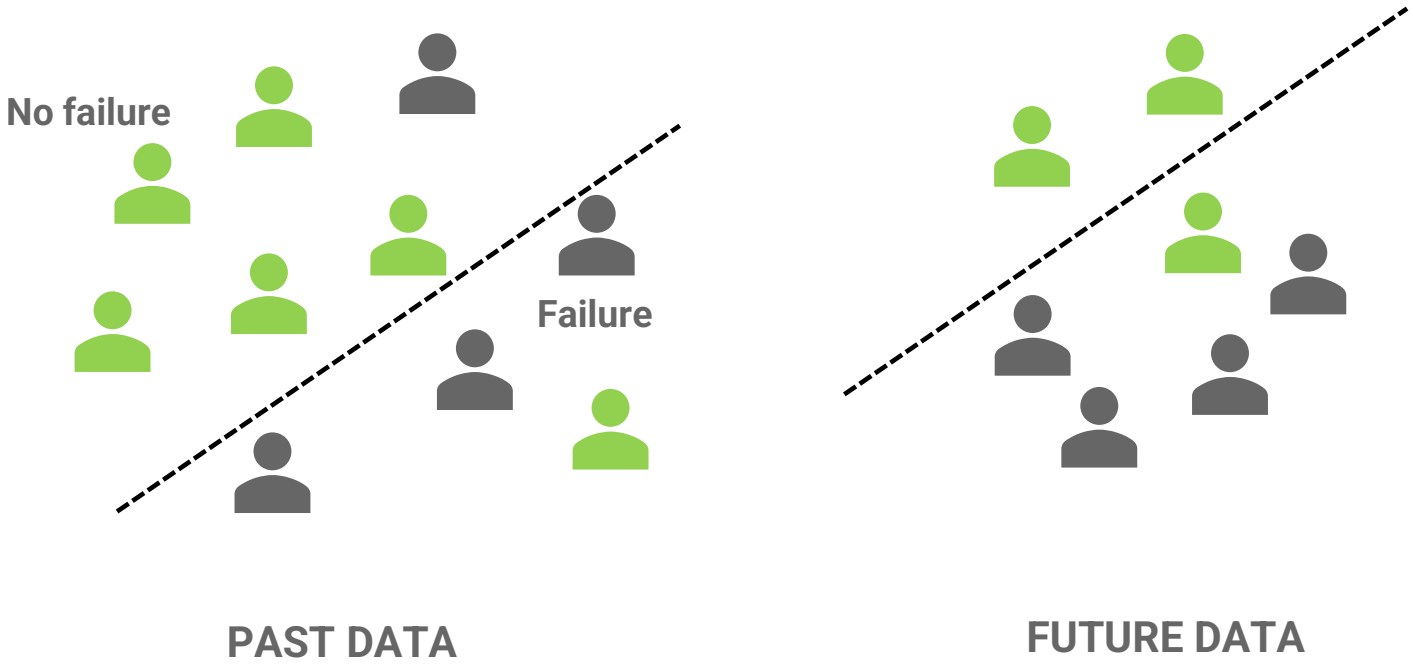
Different models could be generated.



Classification: how to build the model



Classification: how to build the model



From probabilities to class labels

Most algorithms generate a score which is a proxy of the probability of belonging to a given class.

	true_class	pred_class	no	yes
1	*no*	*no*	*1.00000000*	*0.00000000*
2	*no*	*no*	*1.00000000*	*0.00000000*
3	*yes*	*yes*	*0.13333333*	*0.86666667*
4	*no*	*no*	*0.84615385*	*0.15384615*
5	*no*	*no*	*1.00000000*	*0.00000000*
6	*yes*	*yes*	*0.38461538*	*0.61538462*
7	*no*	*no*	*0.92307692*	*0.07692308*
8	*yes*	*yes*	*0.00000000*	*1.00000000*
9	*no*	*no*	*0.84615385*	*0.15384615*

The cutoff value can be properly tuned to effectively classify unbalanced datasets.

Fit a classification model, generate the probability of a test example being in the “yes” class and then play with the cutoff point at which you draw the line between “yes” and “no”.

How would you assess the quality of a model?

pollev.com/andreamor103

Evaluating a classification model

How can we evaluate the quality of a classification model?

Let's introduce the following LOSS FUNCTION

$$L(y_i, f(\mathbf{x}_i)) = \begin{cases} 0 & \text{if } y_i = f(\mathbf{x}_i) \\ 1 & \text{otherwise} \end{cases}$$

and compute the accuracy…

$$\text{acc}_A(\mathcal{S}) = 1 - \frac{1}{m}\sum_{i=1}^{m} L(y_i, f(\mathbf{x}_i))$$

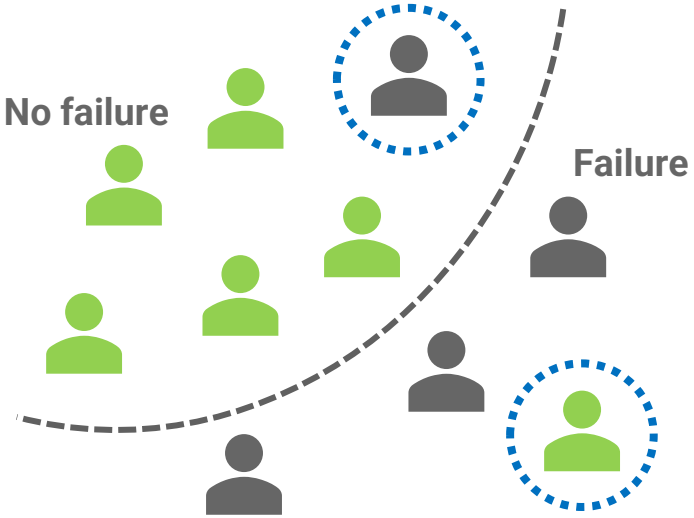
Empirical error

Evaluating a classification model

- **Accuracy** = $\frac{TP+TN}{TP+FP+TN+FN}$

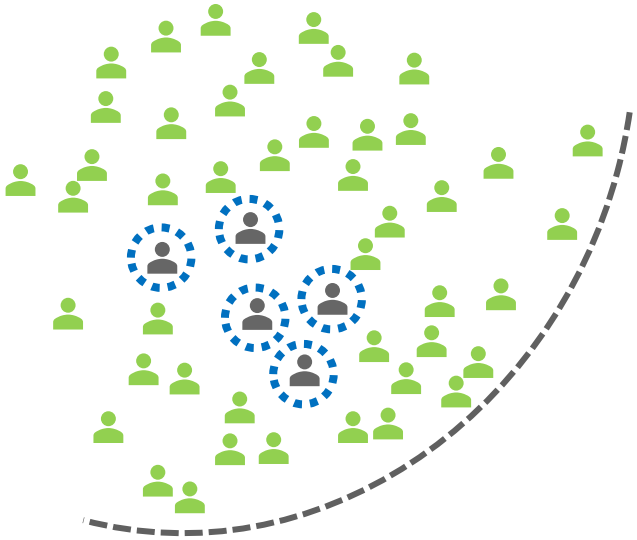
Actual value \ Prediction outcome	0	1
0	True Negative	False Positive
1	False Negative	True Positive

Is accuracy enough?



Accuracy = 80%

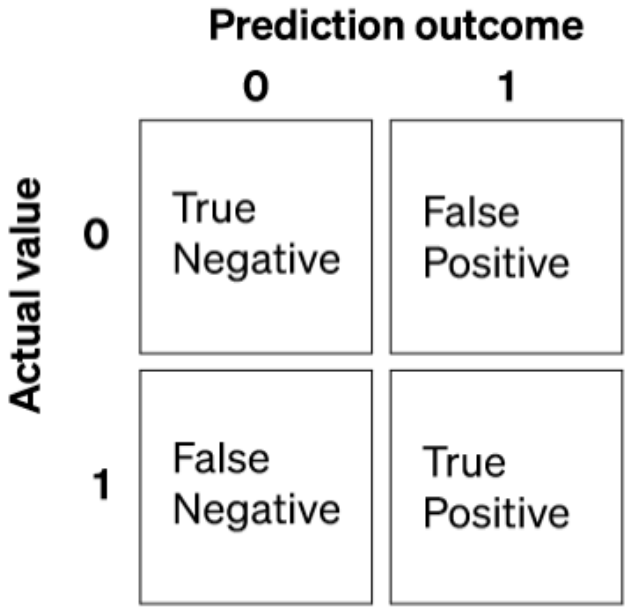
Error = 100 − Accuracy = 20%



Accuracy = 90%

Error = 10%

Evaluating a classification model



- **Accuracy** = $\frac{TP+TN}{TP+FP+TN+FN}$
  When: balanced problems, equal costs of FN and FP.
  When NOT: imbalanced problems.
- **Recall** (True Positive rate)= $\frac{TP}{FN+TP}$
  "proportion of true positives among actual positive"
  When: you want to be sure to catch all the 1s, at the cost of having false alerts.
- **Precision** (Positive Predictive value) = $\frac{TP}{TP+FP}$
  "proportion of true positives among positive predictions"
  When: you want to be sure that all that you predict as 1, are indeed 1, e.g., false alerts are costly.
- Precision and Recall are typically evaluated together.

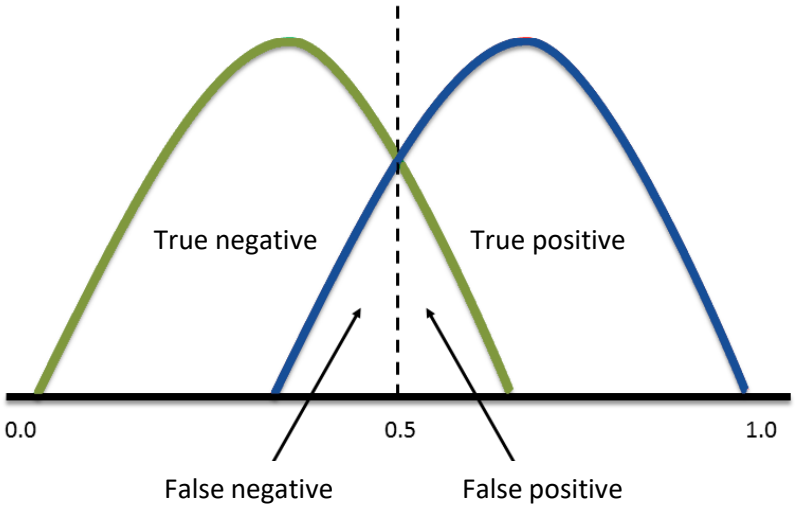
Evaluating a classification model

Actual value \ Prediction outcome	0	1
0	True Negative	False Positive
1	False Negative	True Positive

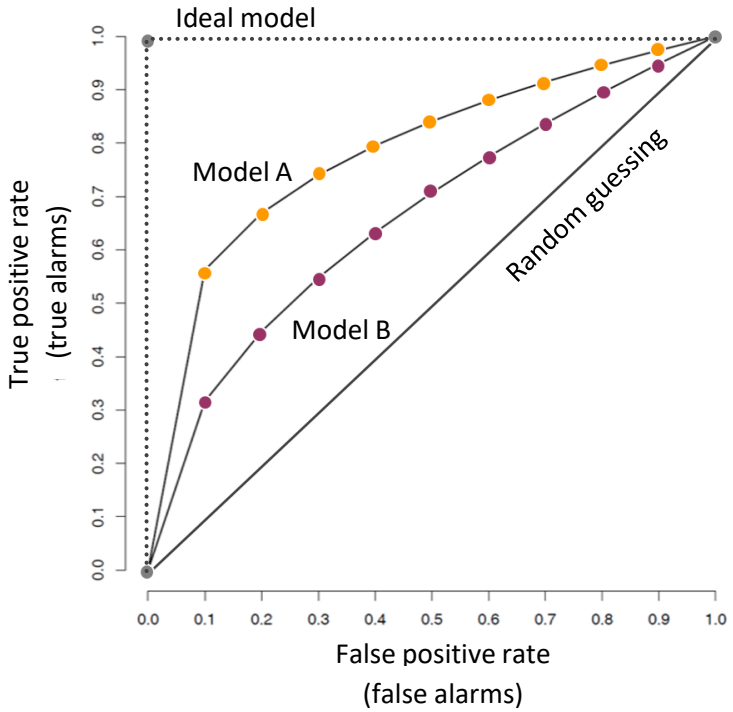
- **Geom. mean**=$\sqrt{\text{Precision} \times \text{Recall}}$
- **F-score**=$\frac{(1+\beta^2)}{\beta^2} \frac{1}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = (1+\beta^2) \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) \cdot \text{Recall}}$
- **F1**: harmonic mean of Prec. and Recall
  $= \frac{2TP}{2TP+FP+FN}$
  Frequently the to-go metric.
- **F2**: 2x emphasis on Recall
  When: False Negatives are more costly.

ROC Curve

Any probability threshold is going to produce true positives, false positives, true negatives and false negatives.



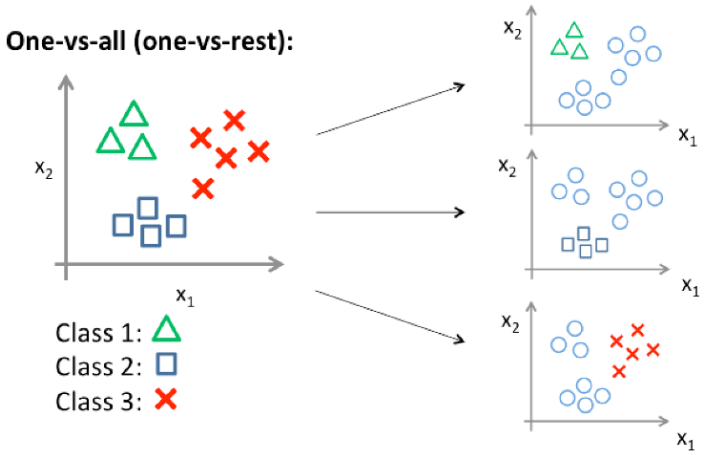
ROC analysis detects possibly optimal models and discard suboptimal ones independently from the class distribution.



A ROC curve can be used to select a threshold for a classifier according to the specific needs (i.e., maximize TP rate or minimize FP rate).

Does classification need to be binary?

1. **One-vs-Rest** We perform $|H|$ different binary classifications: one for every class.



1

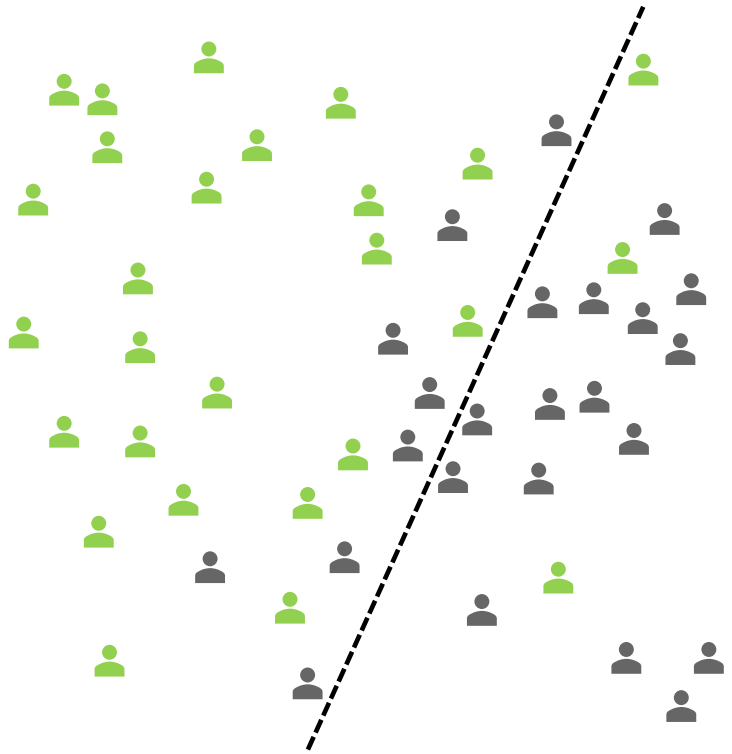
   We decide based on a majority vote.

2. **One-vs-One** We perform $|H|(|H-1|)/2$ binary classifications: one for every pair of classes. We decide based on a majority vote.

Overfitting

door
machine learning group
POLITECNICO DI MILANO

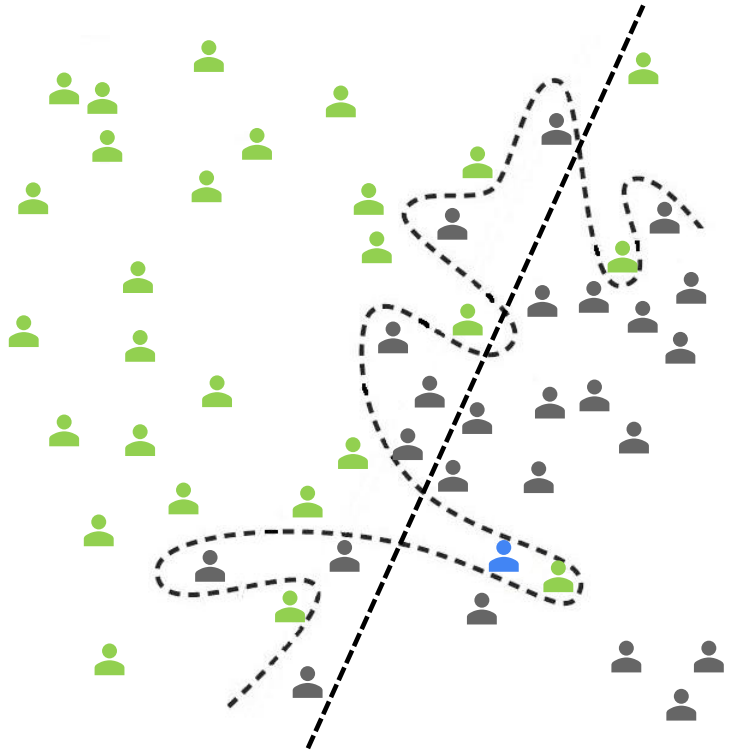
Overfitting occurs when a model learns noise or random fluctuations in the training data so that its performance on new data (generalization ability) is negatively affected.
It is often a result of an excessively complicated model.



Overfitting

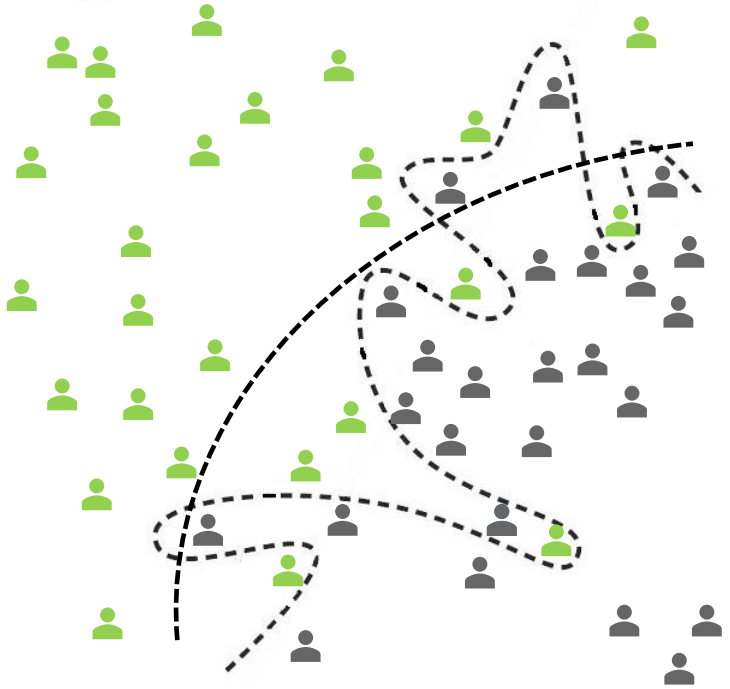
door
machine learning group
POLITECNICO DI MILANO

Overfitting occurs when a model learns noise or random fluctuations in the training data so that its performance on new data (generalization ability) is negatively affected.
It is often a result of an excessively complicated model.

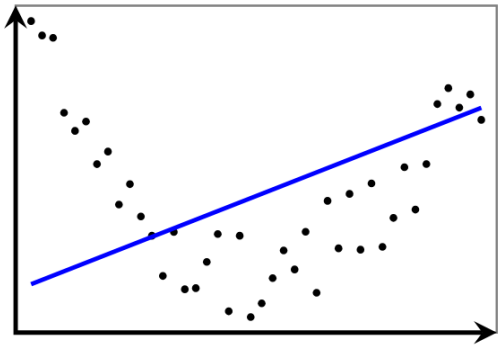


Overfitting

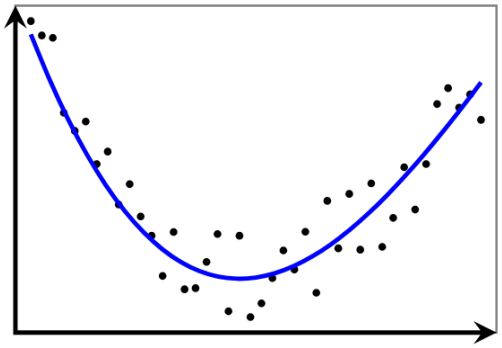
Overfitting occurs when a model learns noise or random fluctuations in the training data so that its performance on new data (generalization ability) is negatively affected.
It is often a result of an excessively complicated model.



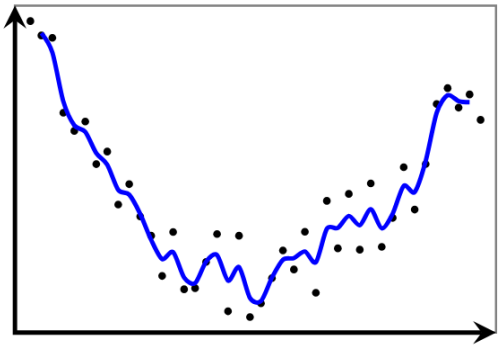
Overfitting



Underfitting

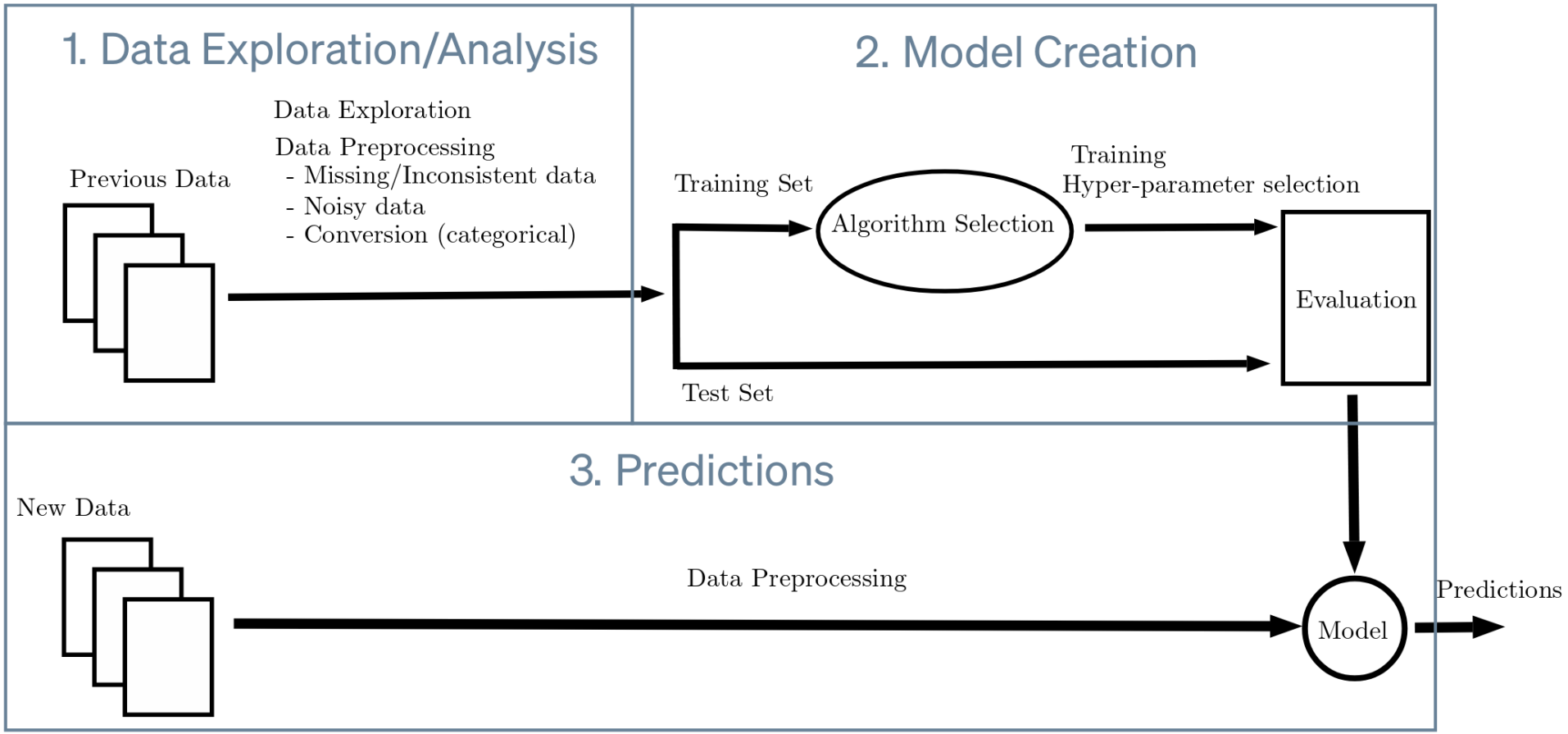


Balance



Overfitting

Machine learning process

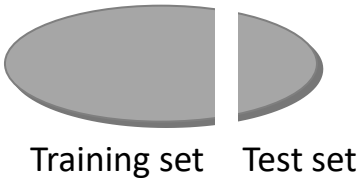


Evaluating a classification model

Accuracy can be estimated by means of four different evaluation schemes:

1 HOLDOUT METHOD

The available examples are divided in two disjoint subsets used for different purposes.

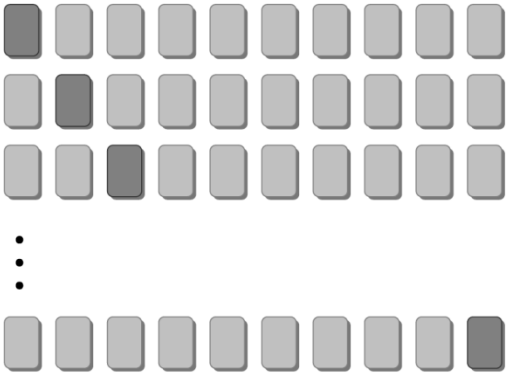


2 REPEATED RANDOM SAMPLING

Holdout method repeated several times.

3 K-FOLD CROSS-VALIDATION

The available examples are divided in two K disjoint subsets.

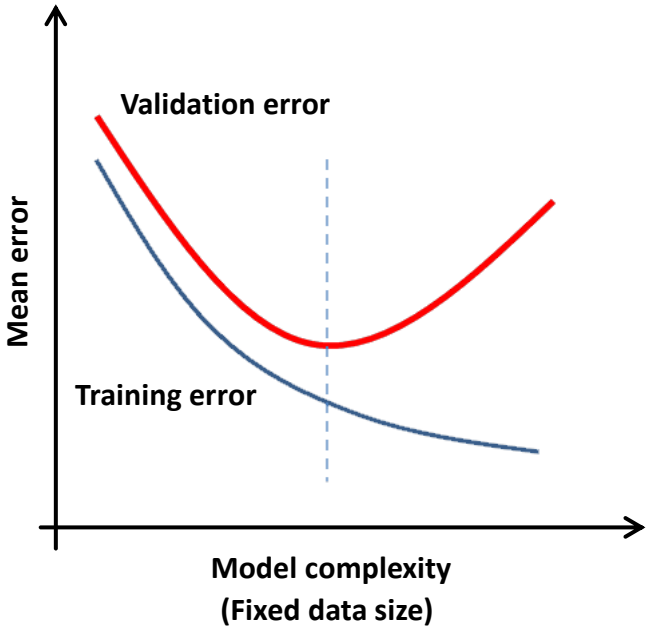


4 LEAVE-ONE-OUT

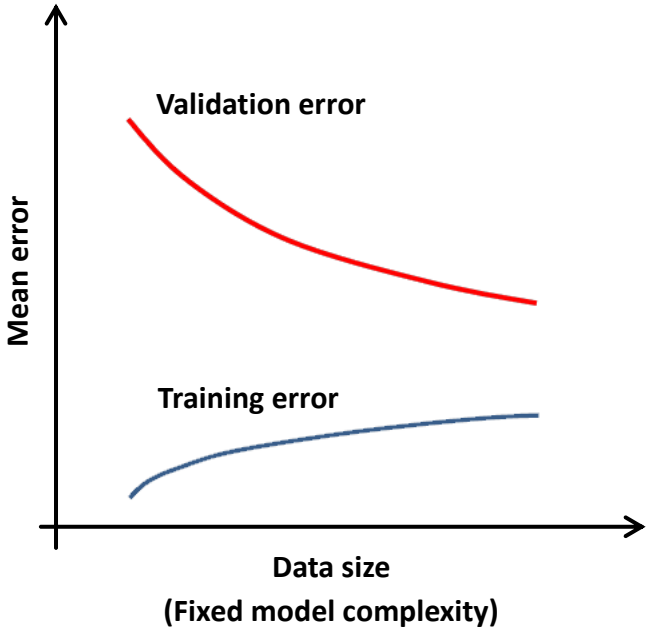
K-fold cross-validation with K = m.

Overfitting

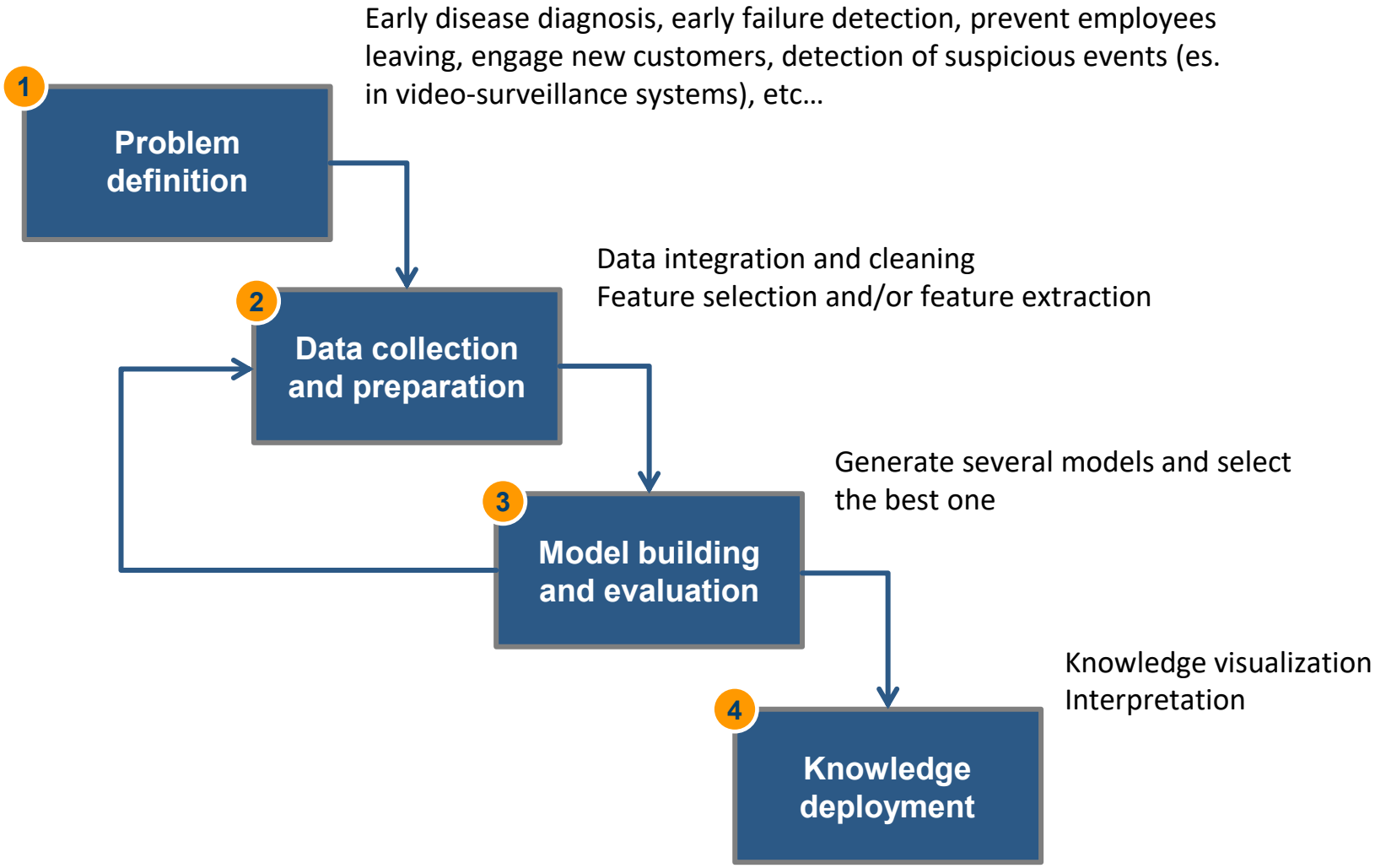
When the size of training data is fixed and the only thing we can choose is the model complexity, then we should choose the model complexity at the point where the cross-validation error is minimal.



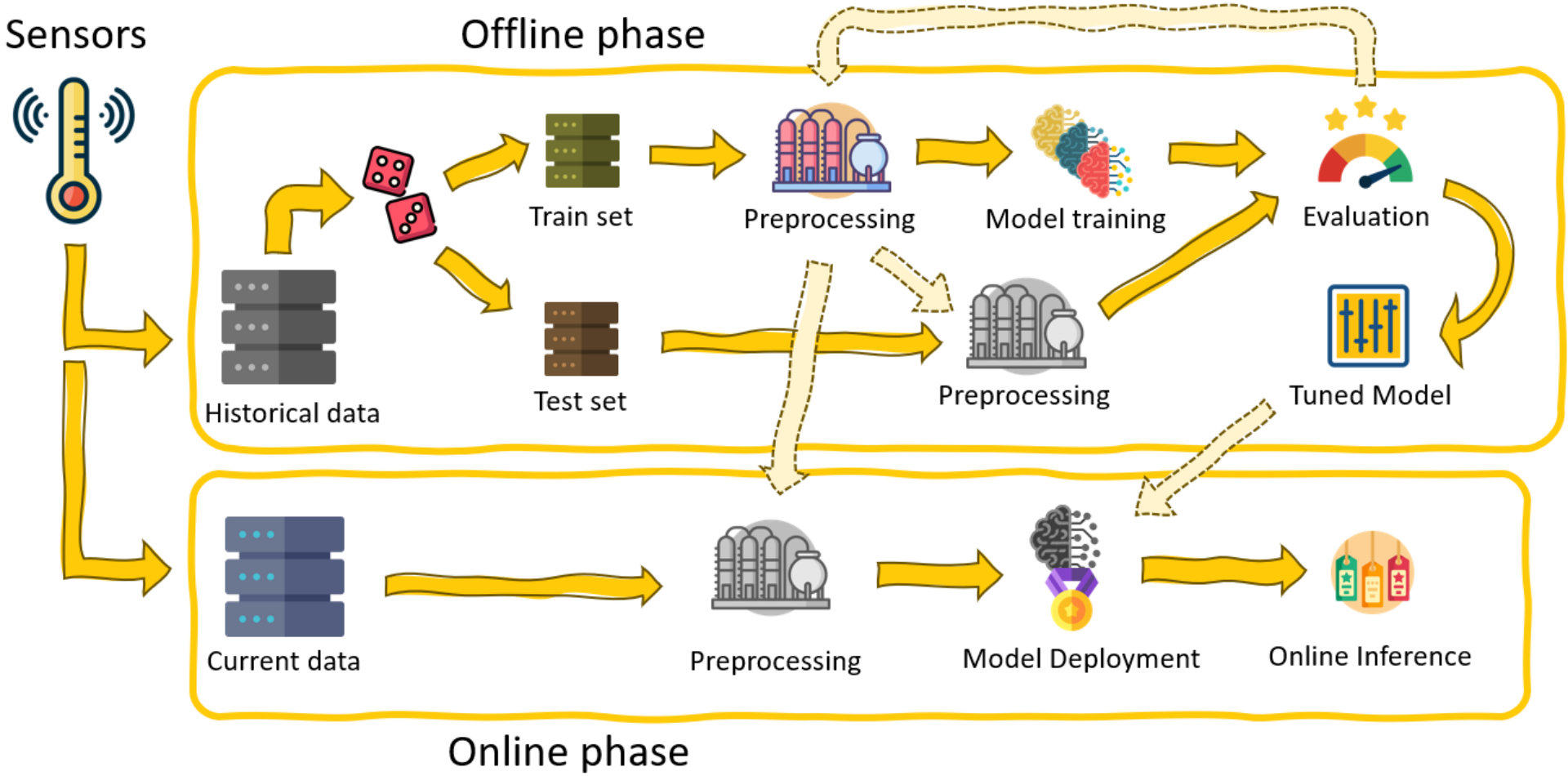
When the underlying data model is in fact complex we cannot reduce the complexity (over-simplified model) but we can collect more training data such that overfitting is less likely to happen (or using bagging).



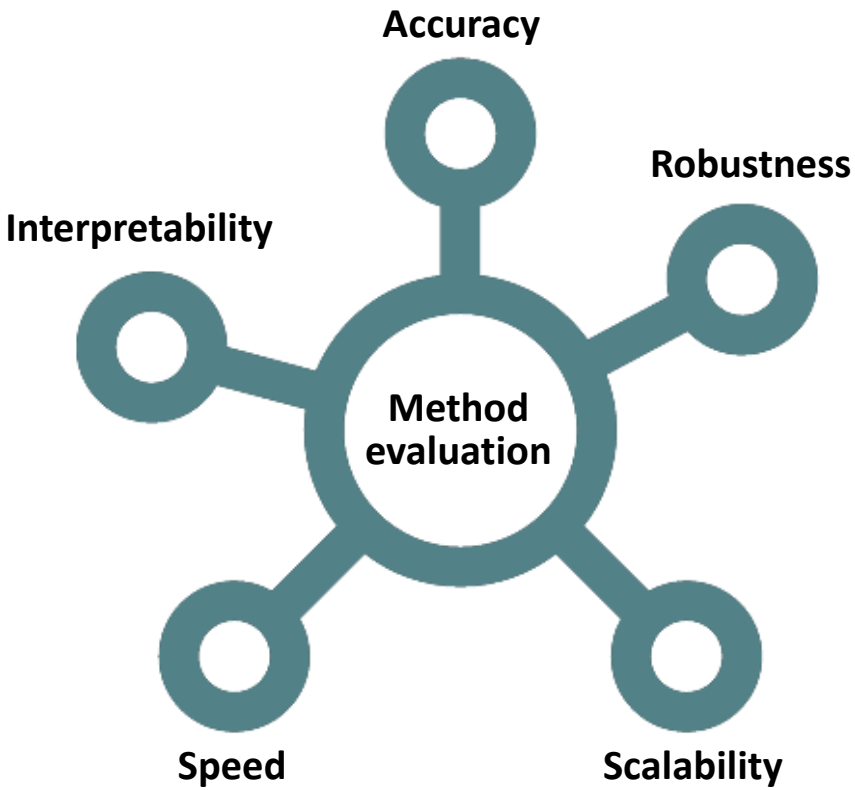
Machine learning process



Machine learning process

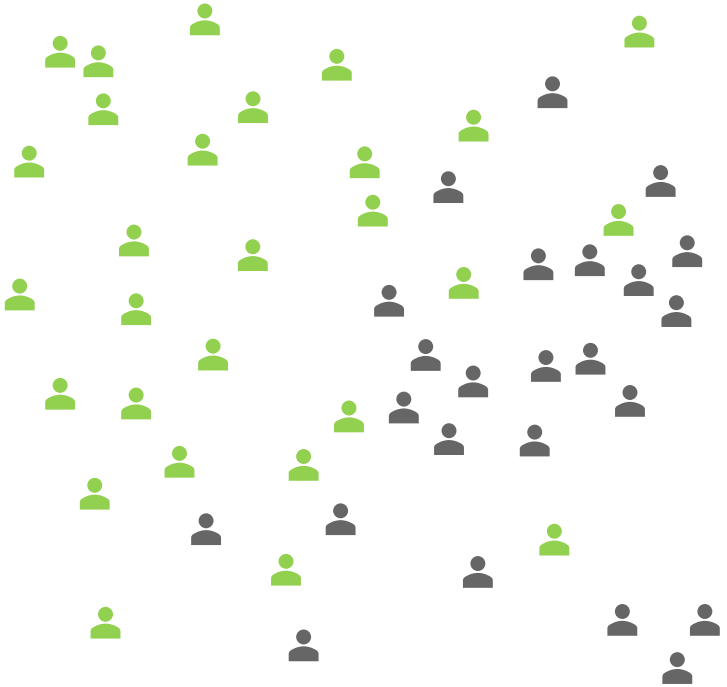


Requirements



How would you build a classification model?

pollev.com/andreamor103

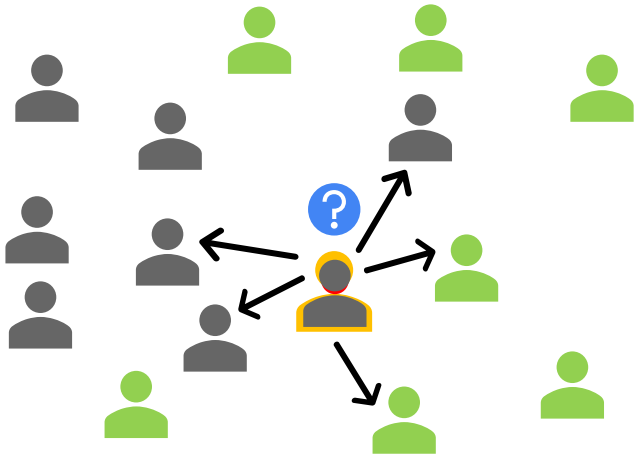


Nearest neighbors classifier (K-NN)

The K-NEAREST NEIGHBORS classifier is a *non-parametric** algorithm used for classification (and regression). It is in the family of *instance-based learning* methods.

A set of training examples must be given. The class of a new example is computed based on the class value of its K nearest neighbors (*majority voting*).

The choice of K affects the accuracy of the model. Rule of thumb: k = radq(m).



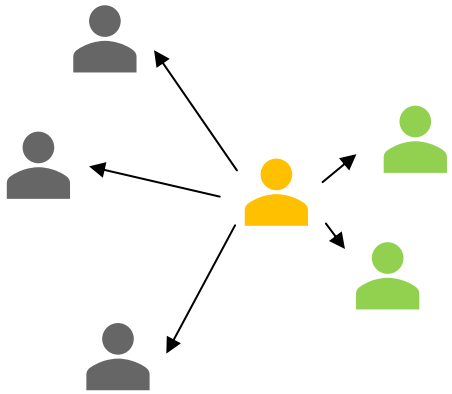
*No assumptions on the distribution from which the training examples are drawn.

Nearest neighbors classifier (K-NN)

Some comments:

1. **Conceptually simple**
2. **It works well with few examples and/or in low dimensions for complex decision surfaces**
3. **Classification might be slow (computationally intensive)**
4. **It suffers from the *curse of dimensionality*.***

   In high dimensions the ratio between the nearest and the farthest point approaches 1 → points become uniformly distant from each other and any kind of distinction becomes meaningless.
5. **Implicit assumption: the class of a new example is close to the classes of its neighbors**

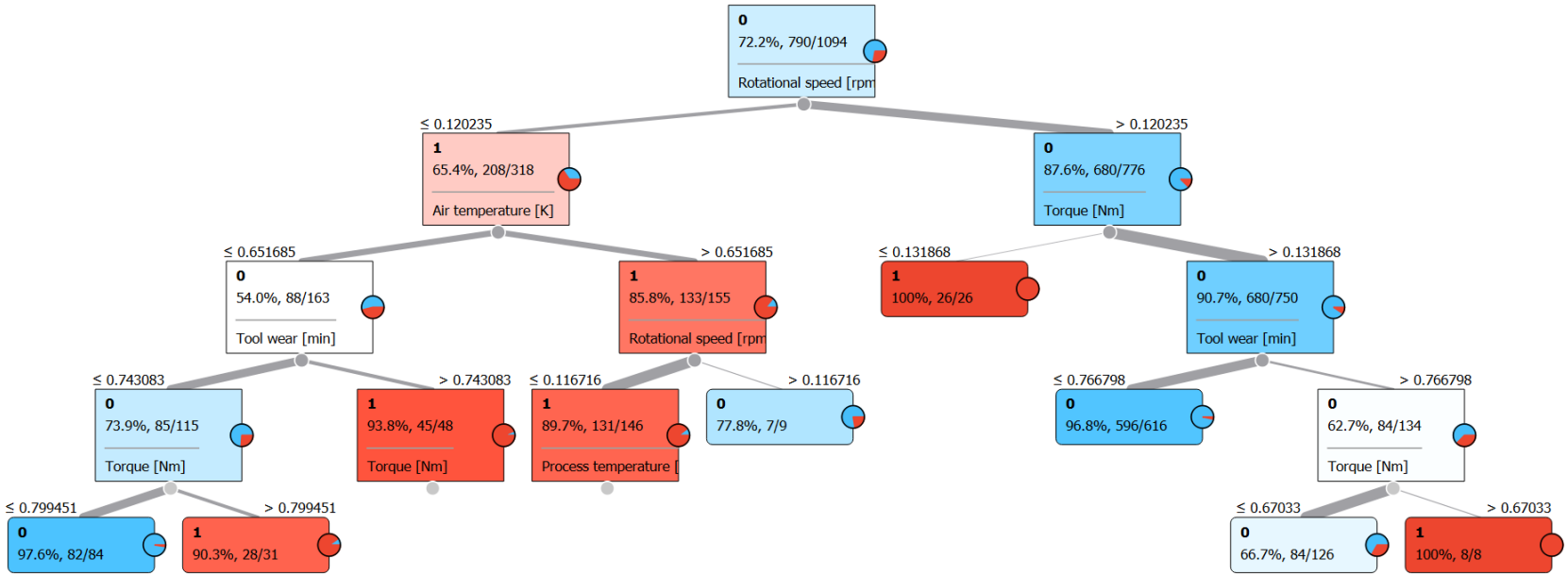


*With an increase in the dimensions the available data become sparse. The amount of data should grow (exponentially) with dimensions.

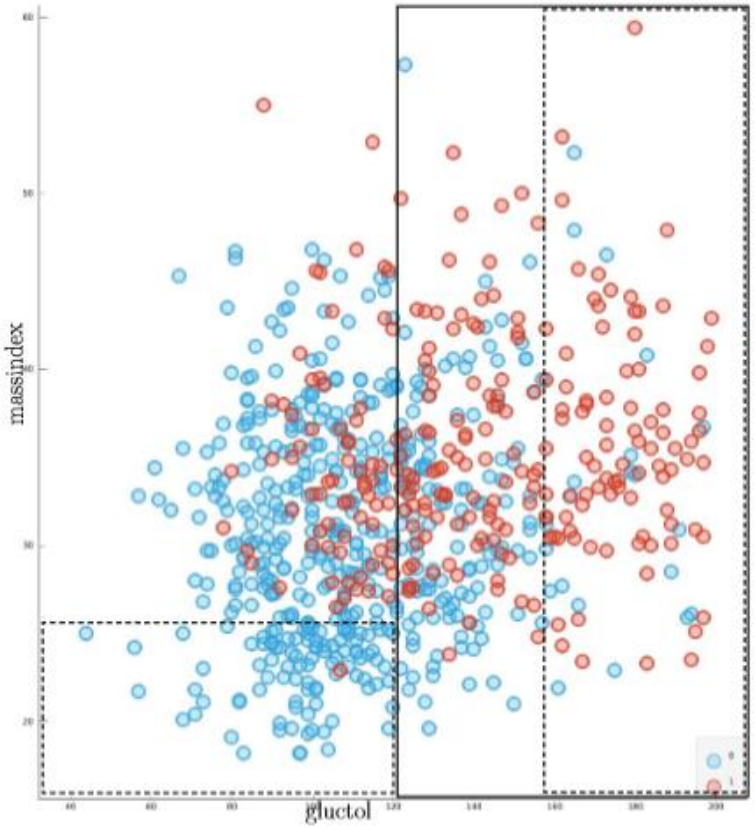
Classification Trees

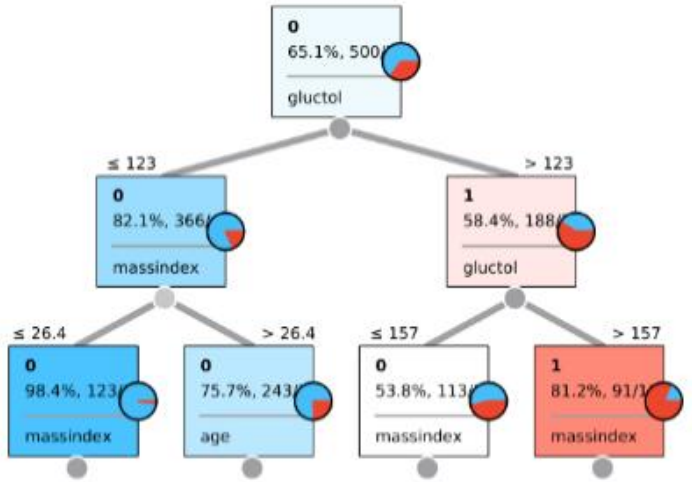
CLASSIFICATION TREES resort to an iterative heuristic procedure based on a *divide-and-conquer* scheme

1. All examples are included in the root node
2. The examples in each node are divided according to a test based on the value of one or more variables
3. A node becomes a leaf if it satisfies one of the stopping conditions
4. Each leaf is labeled according majority voting



Classification Trees



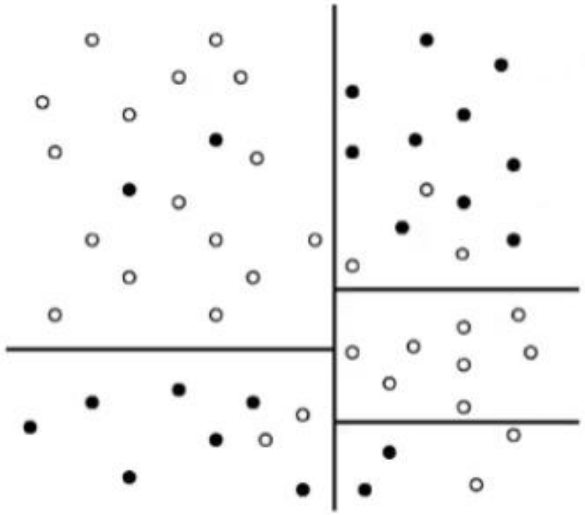


Tree

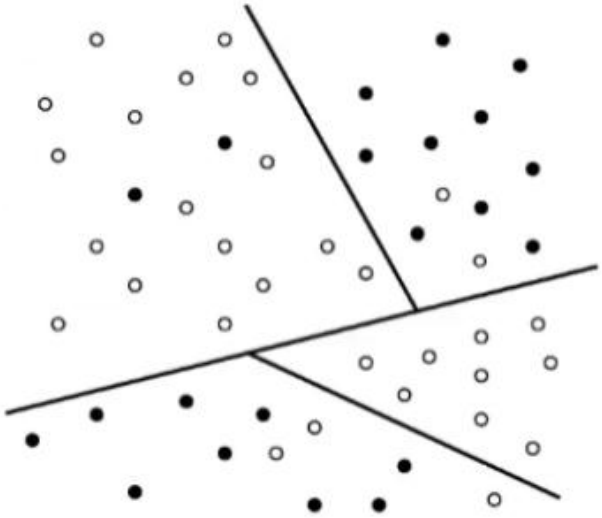
types

- Binary tree (zero/two descendants)
- General trees
- Uni-variate tree ($X_j < b$)
- Multi-variate tree ($\sum_{j=1}^{n} w_j x_j < b$)

Classification Trees

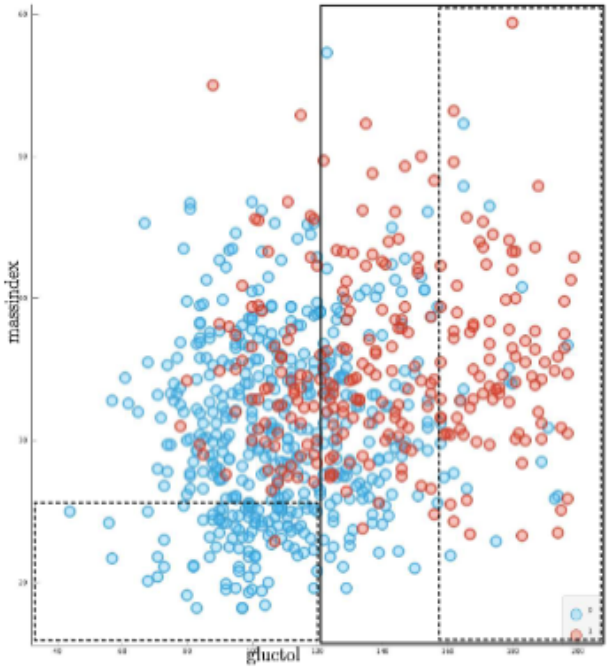


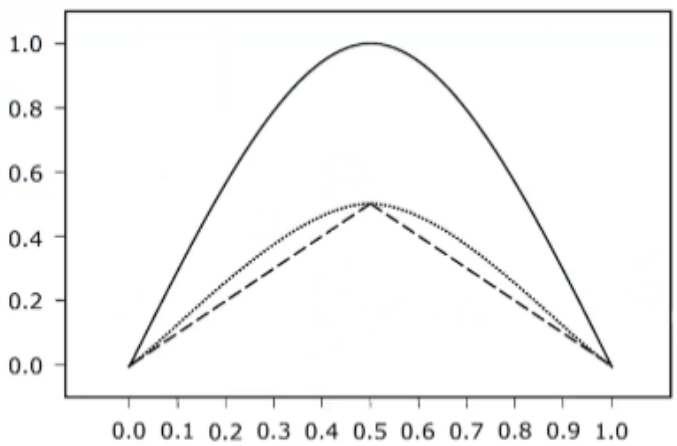
classification by an
axis parallel tree



classification by an
oblique tree

Classification Trees





- **Entropy index:**
- **Gini index:**
- **Miss-classification index:**

Split criteria

$$-\sum_{h=1}^{H} f_h \log_2 f_h$$

$$1 - \sum_{h=1}^{H} f_h^2$$

$$1 - \max_h f_h$$

$$\text{Information gain} = H_p - \sum_{c \in C} \frac{|N_c| H_c}{|N|}$$

Classification Trees

door
machine learning group
POLITECNICO DI MILANO

Some comments:

1. **Classification rules are easy to interpret and explain**
2. **Implicitly perform variable feature selection (variables used at the top are in general the most important ones)**
3. **Are flexible (able to handle both numerical and categorical data)**
4. **Usually benefit from data discretization**
5. **They tend to overfit the training data unless some mechanisms are employed to avoid over-complex trees (*pruning*).**
   - 5a **Pre-pruning** $\rightarrow$ **Stopping criteria (number of examples, purity of the node)**
   - 5b **Post-pruning** $\rightarrow$ **A node is converted into a leaf if the pruned tree makes fewer error on the pruning set than the original tree.**

Bayesian Classifiers

BAYESIAN CLASSIFIERS are aimed at determining the probability that a given example belongs to a class.

- Bayes Theorem

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})} = \frac{P(\mathbf{x}|y)\,P(y)}{\sum_{l=1}^{H} P(\mathbf{x}|y)\,P(y)}$$

- Maximum a posteriori hypothesis

$$y_{MAP} = \arg\max_{y \in \mathcal{H}} P(y|\mathbf{x}) = \arg\max \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$

Bayesian Classifiers

- Categorical/discrete attributes

$$P(x_j|y) = P(x_j = r_{jk}|y = v_h)$$

$\rightarrow$ empirical frequency of the observed value on the class v_h

- Numerical attribute

$$P(x_j|y) \sim N(\mu_{jh}, \sigma_{jh})$$

$\rightarrow$ assuming Gaussian density with empirical parameters

Naïve Bayes classifier

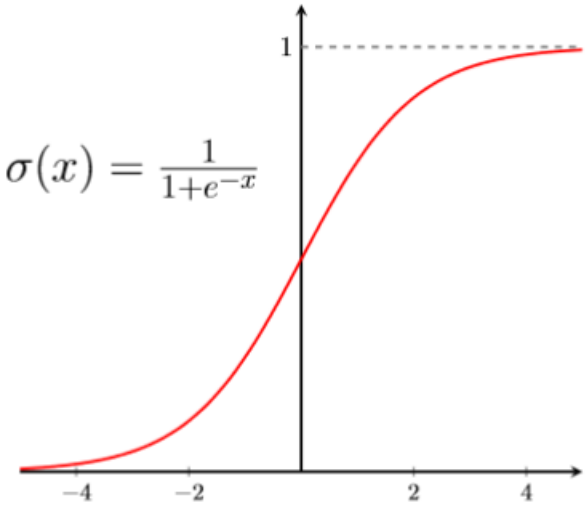
Some comments:

1. **The conditional independence assumption means that interactions between variables can be ignored and are not modeled:**

   - 1a **It employs a very simple hypothesis function (sometimes too simple)**
   - 2a **It ignores some of the information in the training data (no overfitting)**
   - 3a **Requires less training data. Indeed, it has been shown to perform very well with very small training datasets compared to most of the other classifiers**

Bayes Belief Network attempt to overcome such limitations.

Logistic Regression



The model postulates that

$$\log\left[\frac{P(y=1|x)}{P(y=0|x)}\right] = w^T x$$

then if $p = P(y=1|x)$:

$$\frac{p}{1-p} = e^{w^T x}$$

$$\Rightarrow p = P(y=1|x) = \frac{e^{w^T x}}{1+e^{w^T x}} = \frac{1}{1+e^{-w^T x}},$$

$$P(y=0|x) = 1-p = \frac{1}{1+e^{w^T x}} = \frac{e^{-w^T x}}{1+e^{-w^T x}}$$

Logistic Regression

Therefore if we maximizes the likelihood

$$L(w) := P(Y_1 = y_1, \ldots, Y_m = y_m | w, x_1, \ldots, x_m) = \Pi_{i=1}^{n} P(Y_i = y_i | w, x_1, \ldots, x_m).$$

Assuming independence:

$$L(w) := \prod_{i|y_i=1} p(x_i) \cdot \prod_{i|y_i=0} (1 - p(x_i))$$

$$L(w) := \prod_{i=1}^{n} p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

is equivalent to maximize the log likelihood

$$l(w) = \sum_{i=1}^{n} (y_i log(p(x_i)) + (1 - y_i) log(1 - p(x_i)))$$

$$= \sum_{i=1}^{n} (y_i log(\frac{p(x_i)}{1 - p(x_i)}) + log(1 - p(x_i)))$$

Given the quantities defined above

$$l(w) = \sum_{i=1}^{n} (y_i w^T x_i) - log(1 + e^{-w^T x_i}))$$

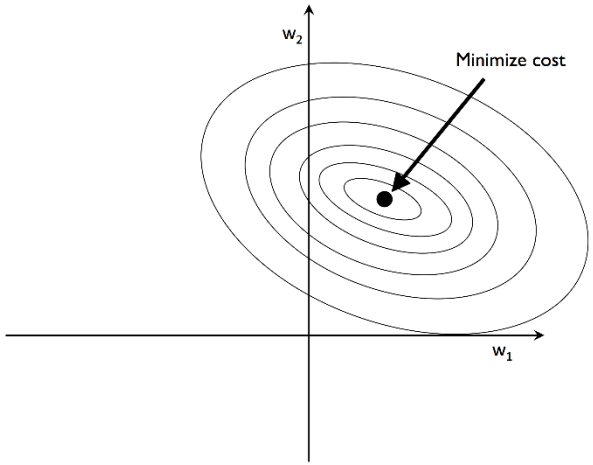
Therefore we aim to resolve the following optimization problem

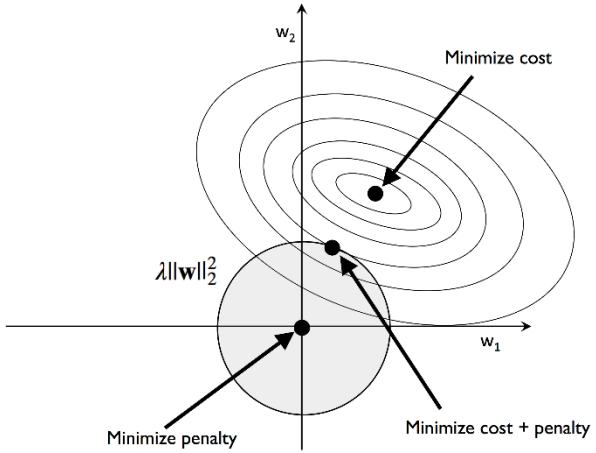
$$\max_w l(w)$$

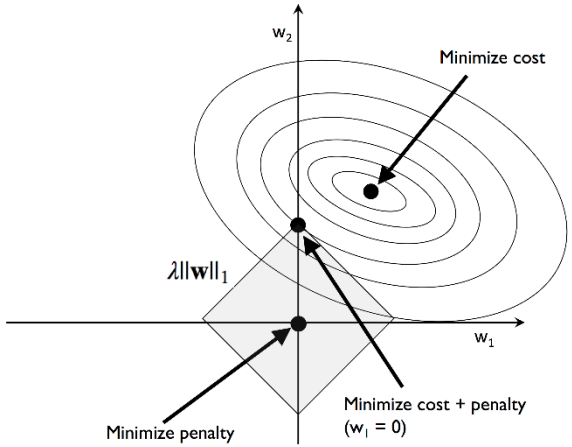
Regularization

Regulation term(s) can be considered

$$\min_w \frac{1}{2}||w||^2 - C \cdot l(w)$$

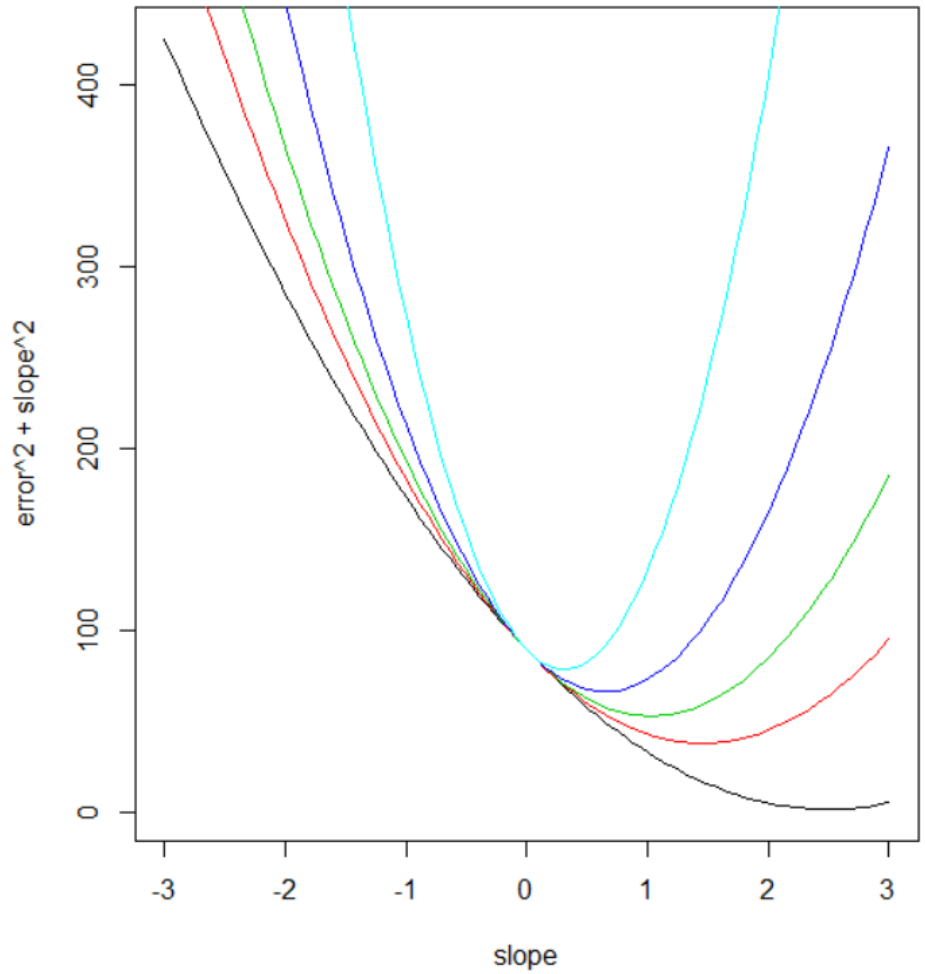


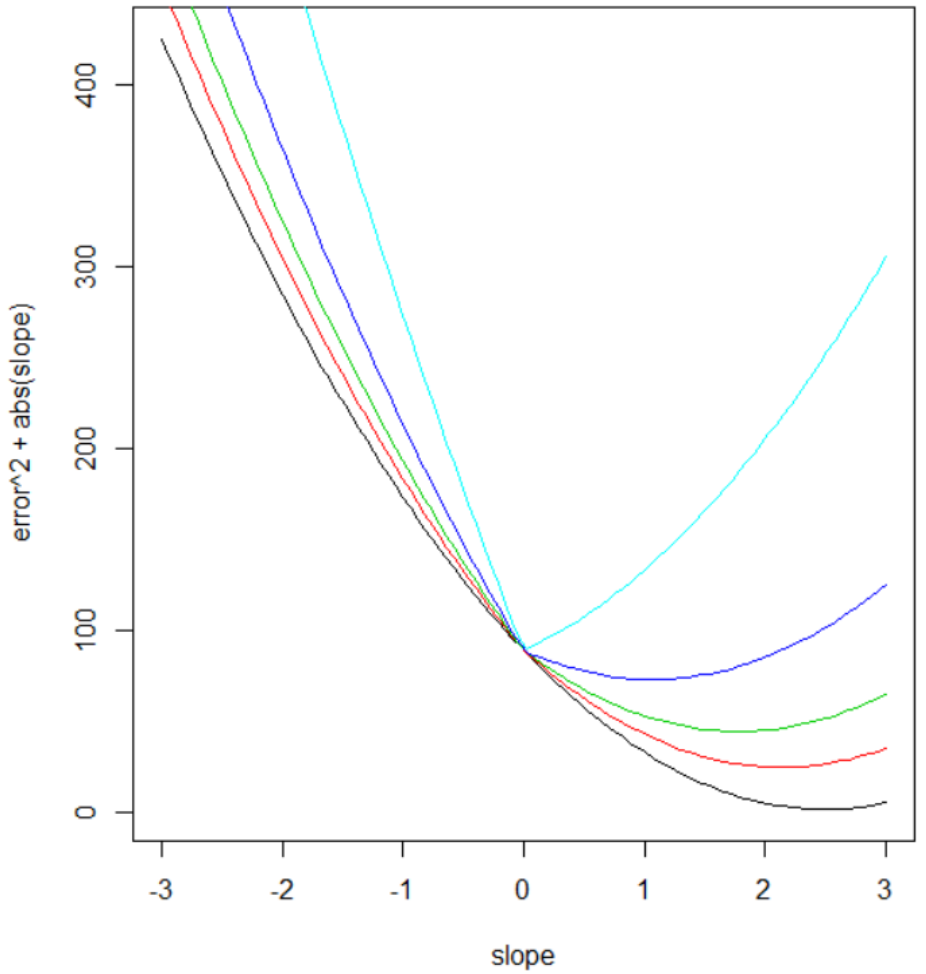




Logistic Regression

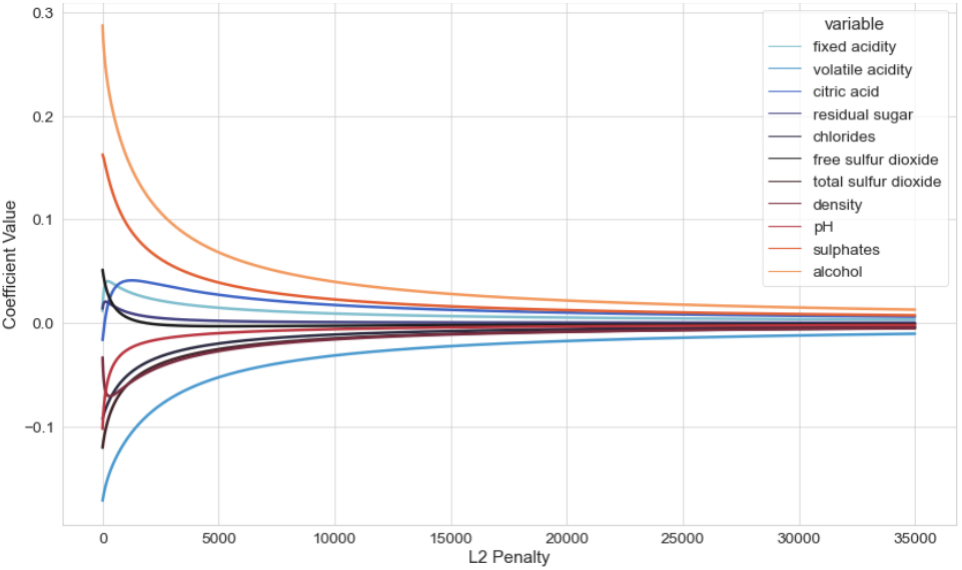
Regularization

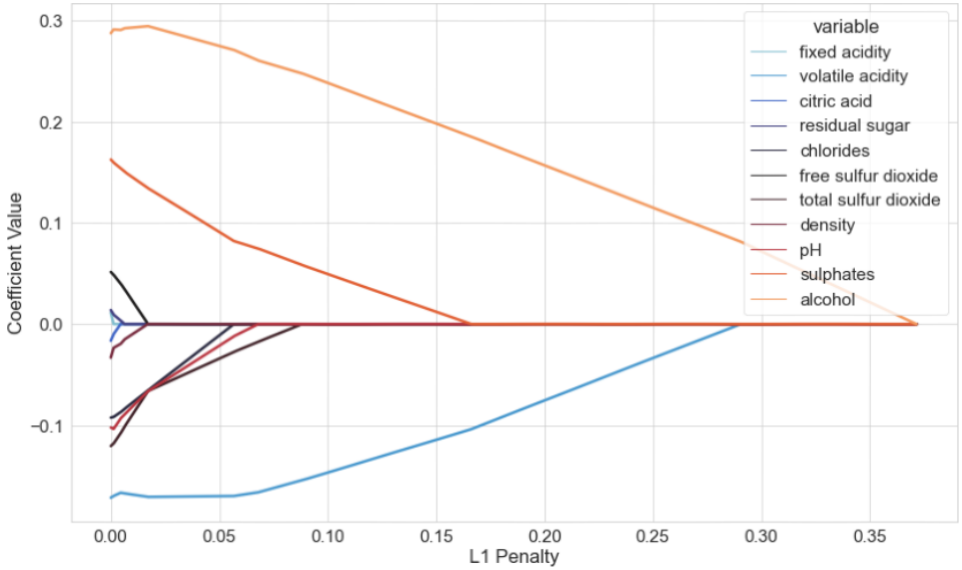




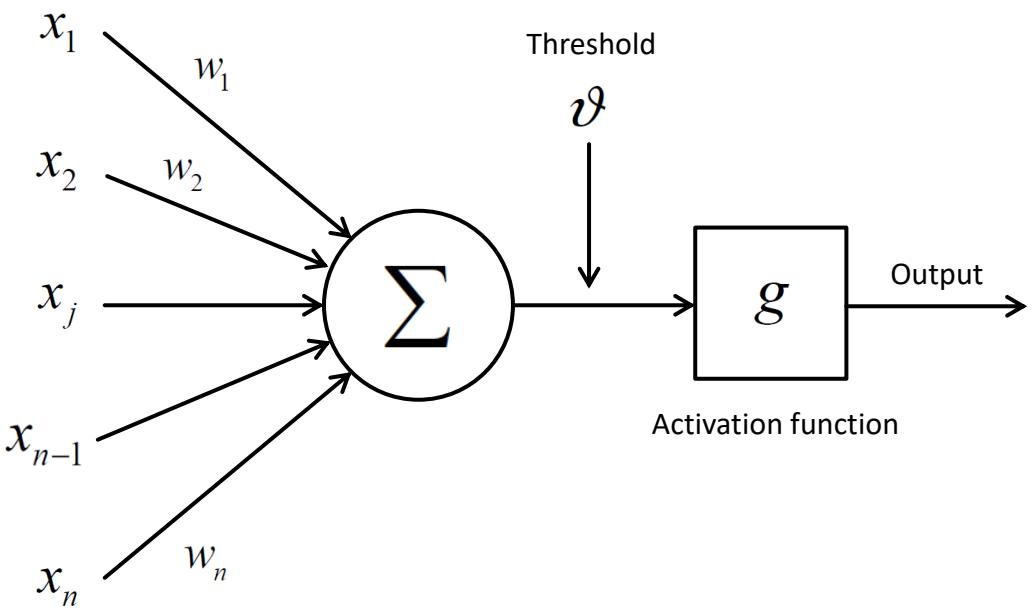
Logistic Regression

Regularization





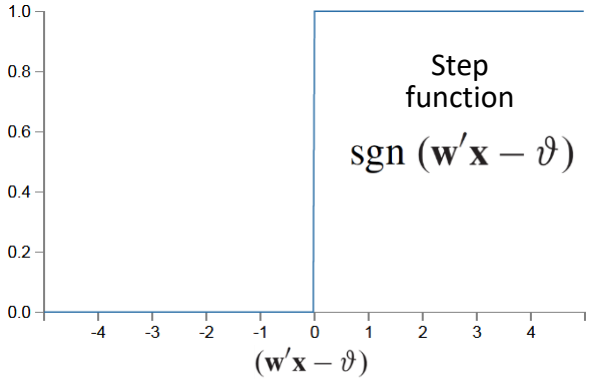
Rosenblatt's Perceptron

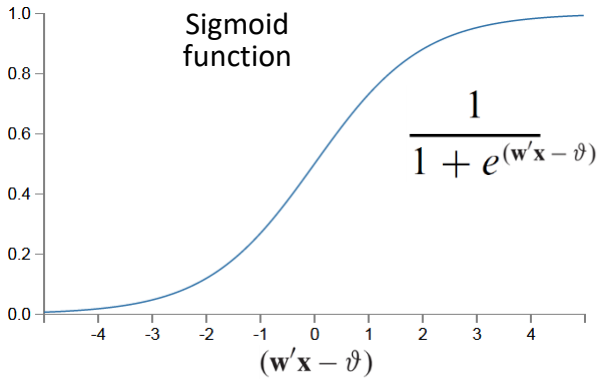


1. Initialize weights and threshold
2. Input an example and compute its output
3. Update weights and threshold according to the entity of the error made

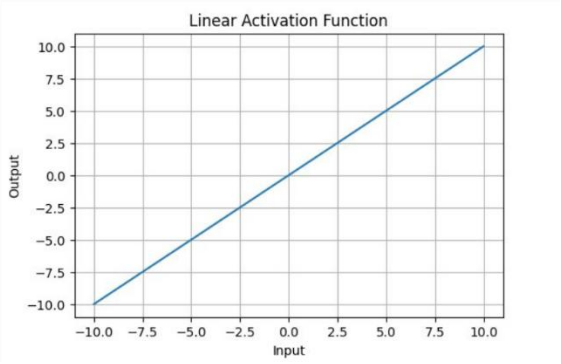
$$\mathbf{W} = \mathbf{W} + \text{error}\ \mathbf{X}$$
$$\vartheta = \vartheta + \text{error}$$

4. Repeat steps 2 and 3 until the error at a given iteration is less than a threshold or a number of iterations have been completed

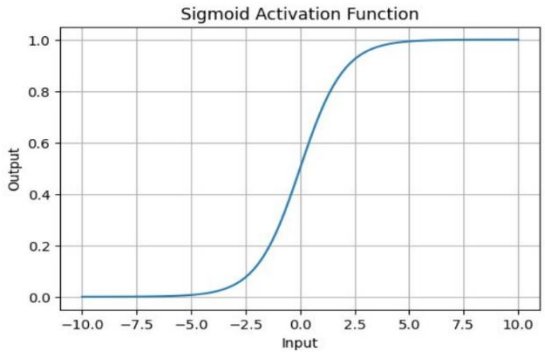




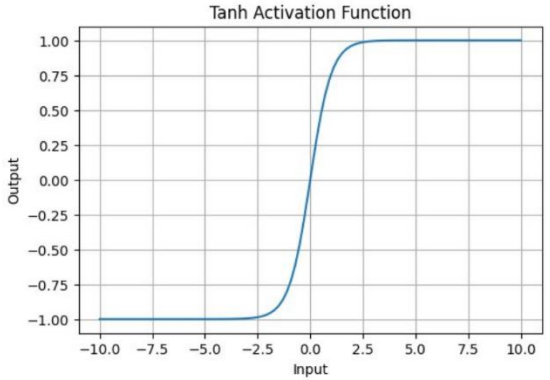
Rosenblatt's Perceptron



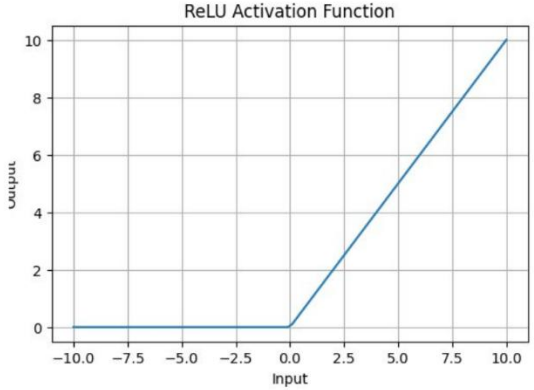
$$h(x) = x$$



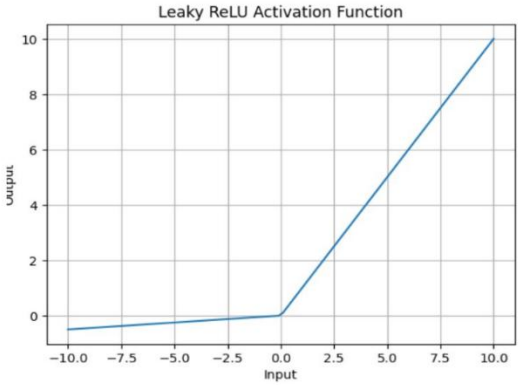
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



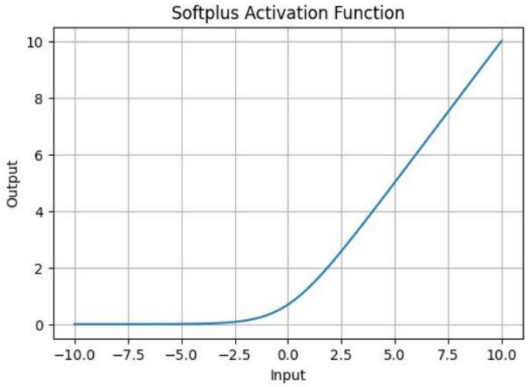
$$tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



$$ReLU(x) = \max(0, x)$$



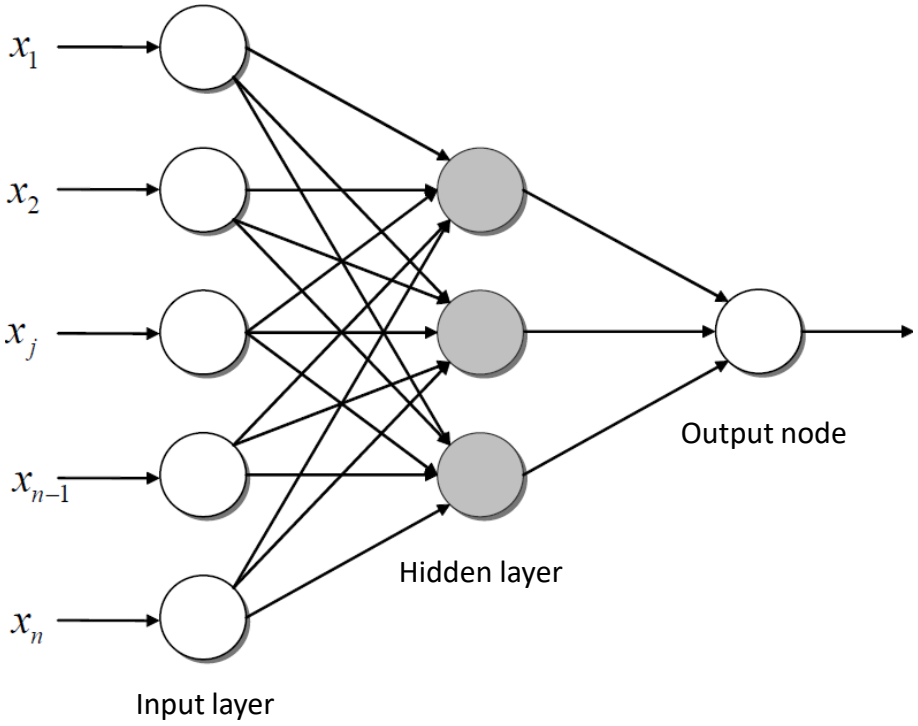
$$Leaky\ ReLU(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



$$f(x) = ln(1 + e^{x})$$

Multi-layer feed-forward neural networks

Each hidden unit can be considered as a single perceptron.



1. Initialize weights and threshold.
2. Input an example, compute its output and the corresponding error.
3. Starting with the output layer propagate the error back to the previous layer and update the weights.
4. Repeat steps 2 and 3 for all the training examples.

Multi-layer feed-forward neural networks

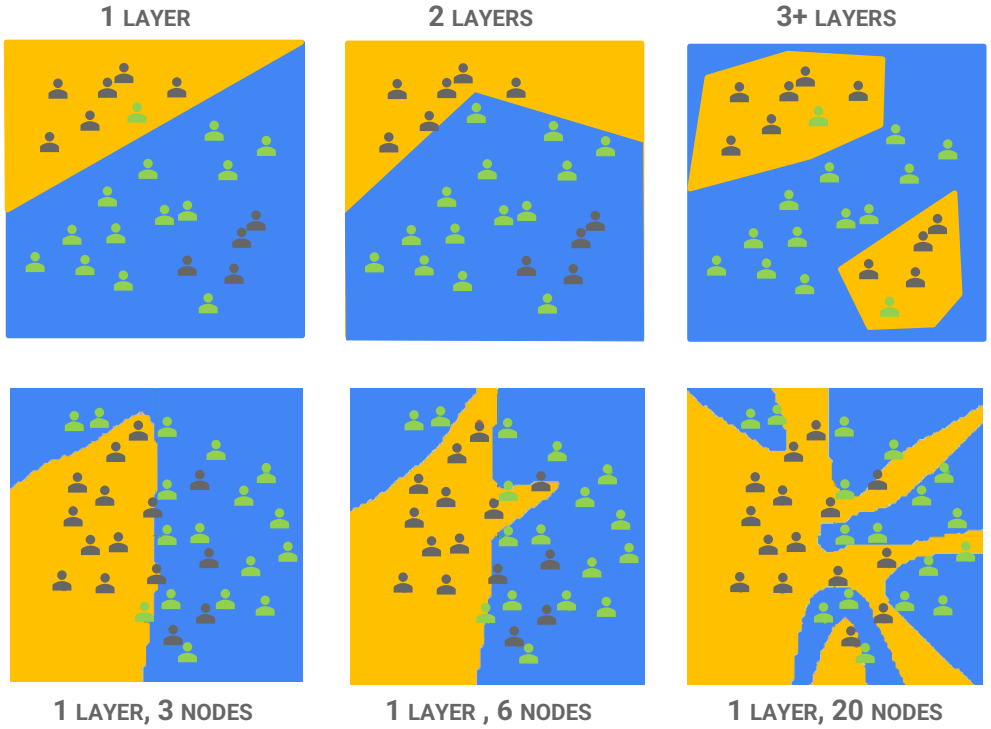
The perceptron generates classification functions in the form of separating hyperplanes.

What happens when the number of hidden layers/nodes increases?

More complex classification regions are obtained!

You can play with this here:

https://playground.tensorflow.org



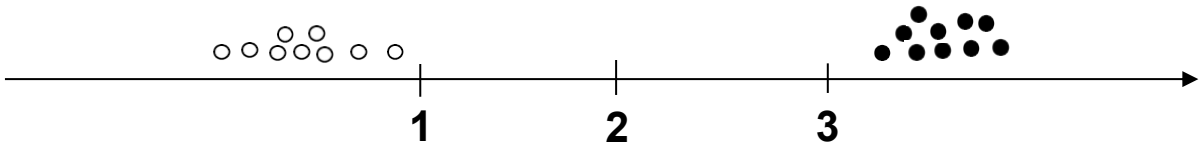
Neural networks

Some comments:

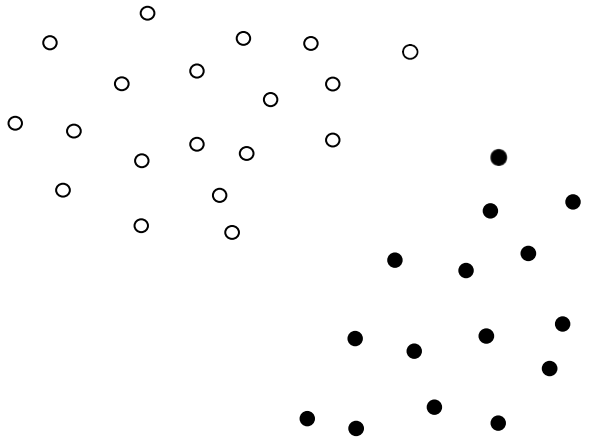
1. **Very effective techniques (deep learning)**
2. **Training a network is an art…and can be extremely expensive**
3. **Interpretability is among the most prominent issues**

Support Vector Machines

Support Vector Machines

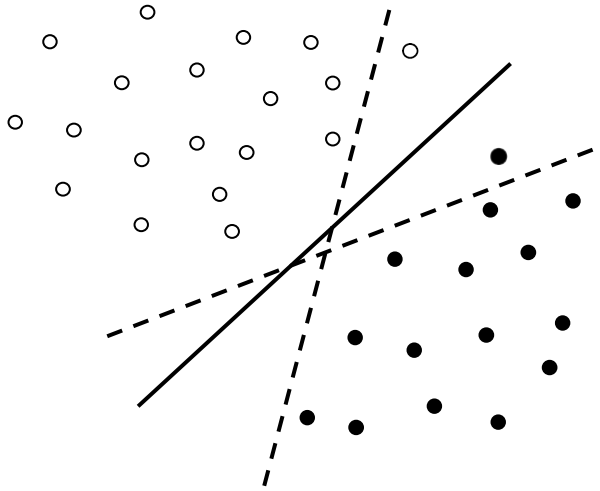


Support Vector Machines



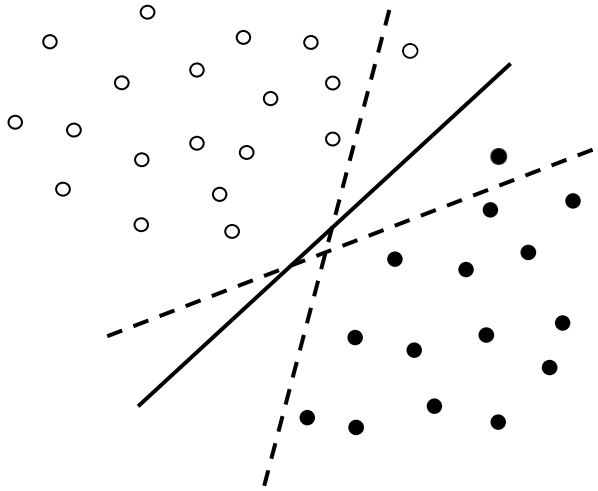
Support Vector Machines

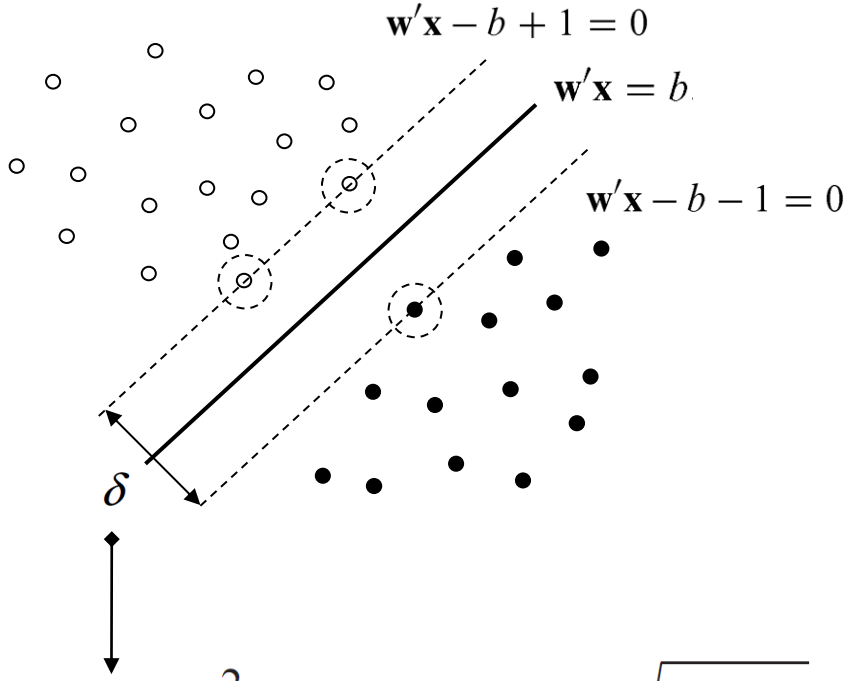
If the examples are linearly separable there exist infinite optimal separating hyperplanes…



Support Vector Machines

If the examples are linearly separable there exist infinite optimal separating hyperplanes…



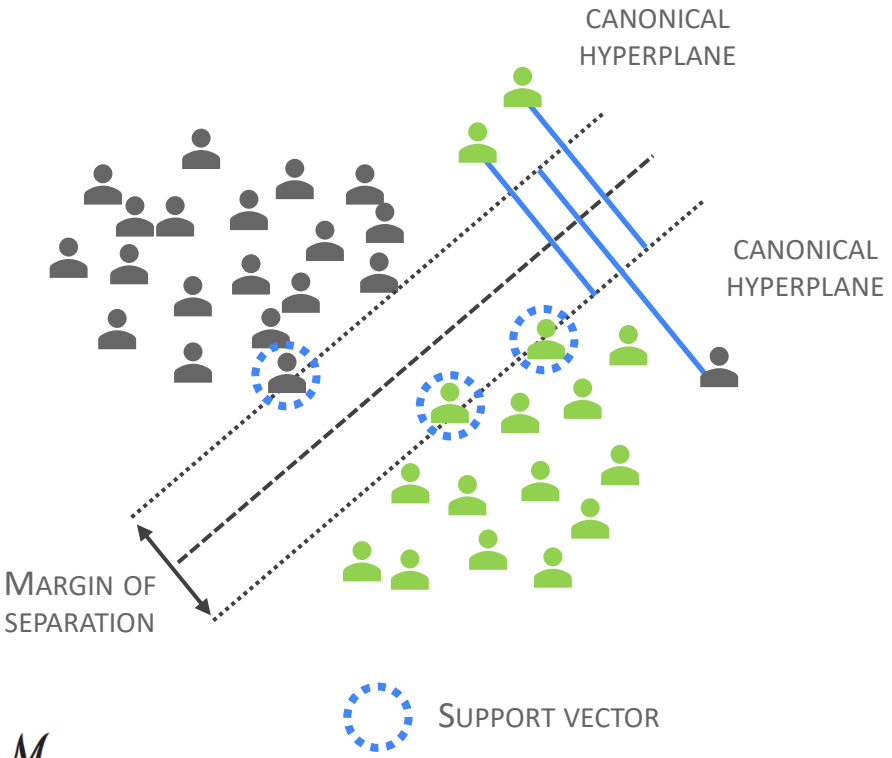


Margin of separation $\rightarrow$ $\delta = \dfrac{2}{\|w\|}$ where $\|w\| = \sqrt{\sum_{j \in \mathcal{N}} w_j^2}$

Support Vector Machines

If the dataset is linearly separable, the optimal separating hyperplane is determined by solving the following optimization problem:

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\|\mathbf{w}\|^2$$
$$\text{s.a} \quad y_i(\mathbf{w}\mathbf{x}_i - b) \geq 1 \quad i \in \mathcal{M}$$



Allowing for soft margins:

$$\min_{\mathbf{w},b,d} \quad \frac{1}{2}\|\mathbf{w}\|^2 + \lambda\sum_{i=1}^{m} d_i$$
$$\text{s.a} \quad y_i(\mathbf{w}\mathbf{x}_i - b) \geq 1 - d_i \quad i \in \mathcal{M}$$
$$d_i \geq 0 \quad i \in \mathcal{M}$$

Support Vector Machines

How do we solve this quadratic problem? Lagrangian multipliers

$$L(\mathbf{w}, \alpha, b) = \frac{1}{2}||\mathbf{w}||^2 - \sum_{i \in N} \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x_i} + b) - 1]$$

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i \in N} \alpha_i y_i \mathbf{x}_i = 0$$

$$\Rightarrow \mathbf{w} = \sum_{i \in N} \alpha_i y_i \mathbf{x}_i \text{ i.e., weight are a linear sum of the samples}$$

$$\frac{\partial L}{\partial b} = -\sum_{i \in N} \alpha_i y_i = 0$$

$$\Rightarrow \sum_{i \in N} \alpha_i y_i = 0$$

$$L(\mathbf{w}, \alpha, b) = \sum_{i \in N} \alpha_i - \frac{1}{2} \sum_{i \in N} \sum_{j \in N} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j$$

Support Vector Machines

What about nonlinear separations?

$$L(\mathbf{w}, \alpha, b) = \sum_{i \in N} \alpha_i - \frac{1}{2} \sum_{i \in N} \sum_{j \in N} \alpha_i \alpha_j y_i y_j \mathbf{x}_i' \mathbf{x}_j$$

$$L(\mathbf{w}, \alpha, b) = \sum_{i \in N} \alpha_i - \frac{1}{2} \sum_{i \in N} \sum_{j \in N} \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i)' \phi(\mathbf{x}_j)$$

Support Vector Machines

What about nonlinear separations?

- **Let's consider a second degree polynomial mapping:**

$$\phi(\mathbf{x}) \overset{\text{e.g.}}{=} \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) \overset{\text{e.g.}}{=} \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}$$

- **This, however, can lead to operations with vectors of very high dimensionality.**
- **But we are only interested in the result of $\phi(\mathbf{x}) \cdot \phi(\mathbf{z})$ -> Kernel trick**
- **Kernel function:**

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$$

Support Vector Machines

For a second degree polynomial mapping:

$$\begin{aligned}\phi(\mathbf{x}) \cdot \phi(\mathbf{z}) &= \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}^T \begin{pmatrix} z_1^2 \\ \sqrt{2}z_1z_2 \\ z_2^2 \end{pmatrix} \\ &= x_1^2z_1^2 + 2x_1z_1x_2z_2 + x_2^2z_2^2 \\ &= (x_1z_1 + x_2z_2)^2 = \left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}^T \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} \right)^2 = (\mathbf{x} \cdot \mathbf{z})^2\end{aligned}$$

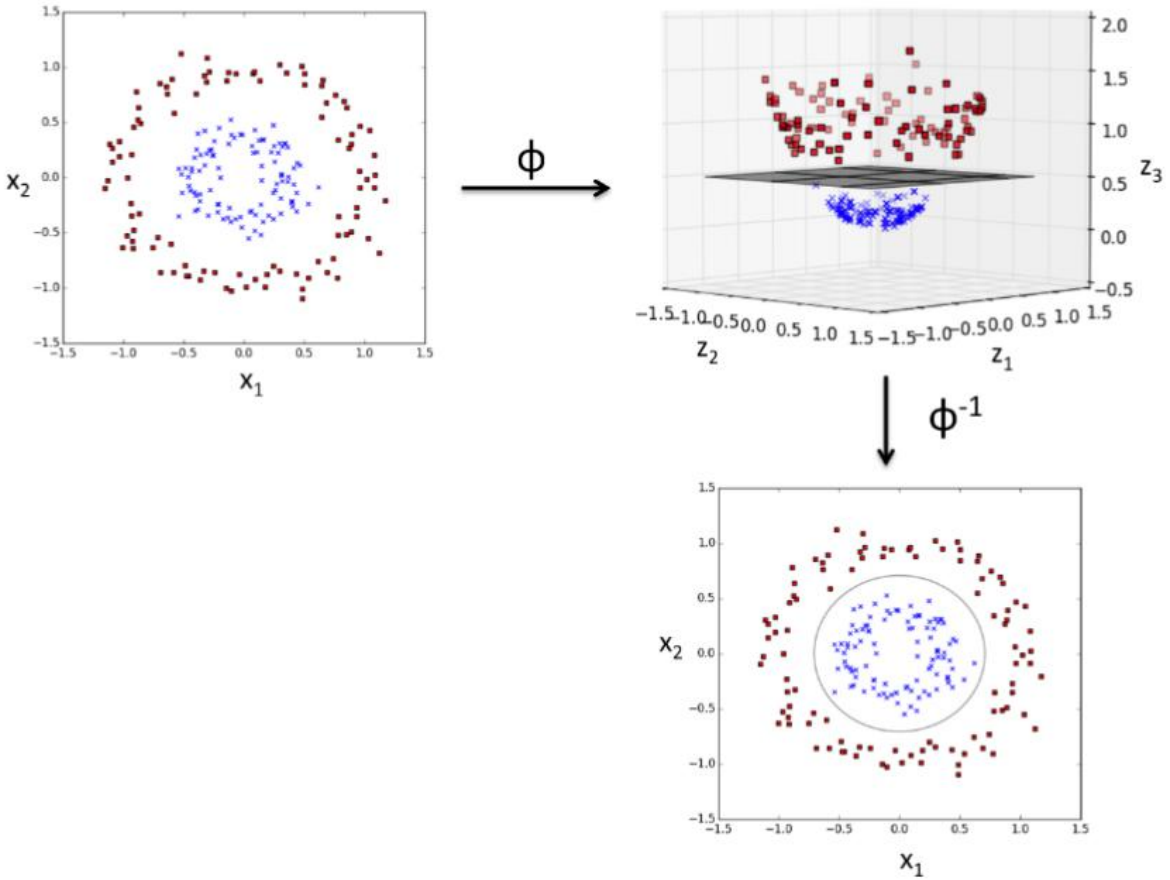
Some well known kernels:

- linear: $x'x$
- poly: $(\gamma x'x + r)^d$
- rbf: $e^{(-\gamma||x-x'||^2)}$
- sigmoid: $\tanh(\gamma x'x + r)$

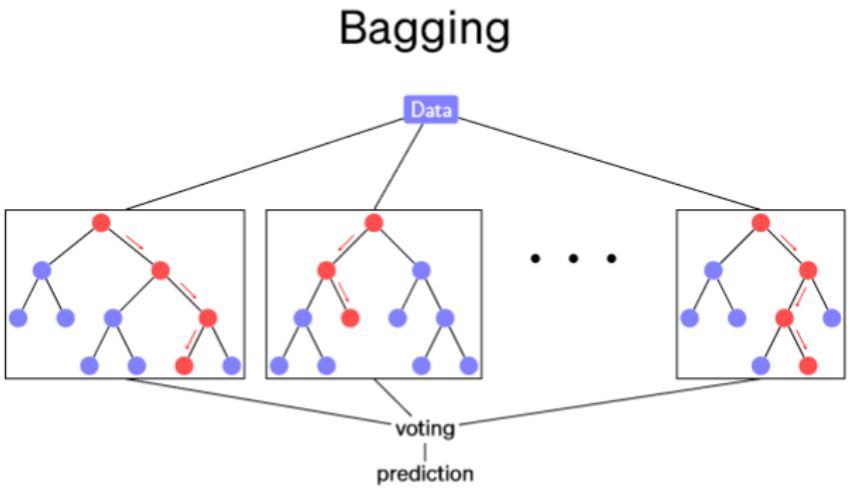
Support Vector Machines

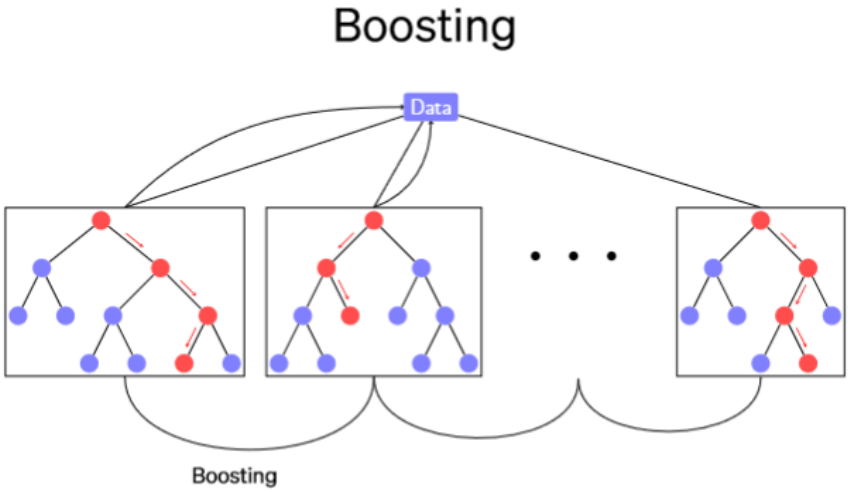
door
machine learning group
POLITECNICO DI MILANO

What about nonlinear separations?



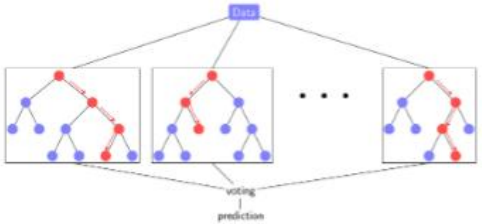
Ensamble methods





Bagging example: Random Forests

Bagging (bootstrap aggregating) is an ensemble framework aimed at improving the stability and accuracy of a classifier. It helps to avoid overfitting.

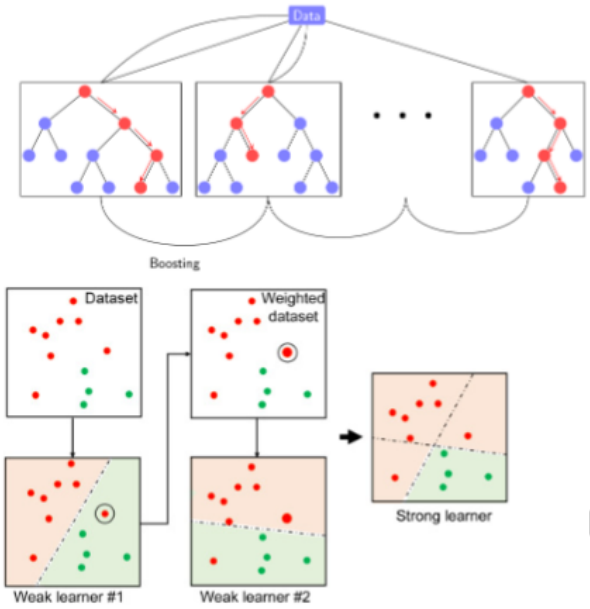


1. Create different (simple) tree models (stumps)
2. Each model is created with a subset of observation/features (~2m/3)
3. We combine the prediction of all trees

Main Parameters

- n_estimators: Number of trees
- max_features: Number of features selected for the split
- bootstrap=False: Use all samples
- Tree parameters

Boosting example: Adaptive Boosting



1. Assign equal weights to observations $w_i^{(0)} = 1/m$
2. For $k = 1, \ldots, K$
   - Select a sample of observations based on the weights.
   - Create the k-th weak learner and compute predictions $x^{(k)}$
   - Compute the model **weighted** error and assign its coefficient:

$$\alpha^{(k)} = \lambda \times \log((1 - error)/error)$$

   - Update sample weights:

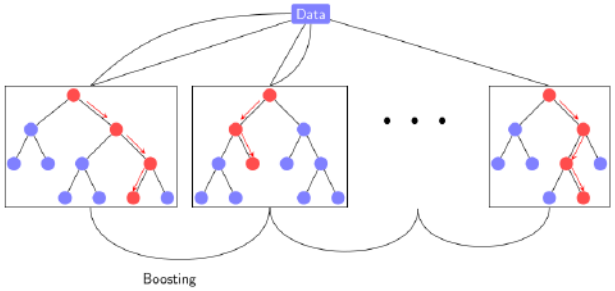
$$w_i^{(k+1)} \propto w_i^{(k)} \times \exp(-\alpha^{(k)} y_i \hat{x}_i^{(k)})$$

3. Final weighted prediction

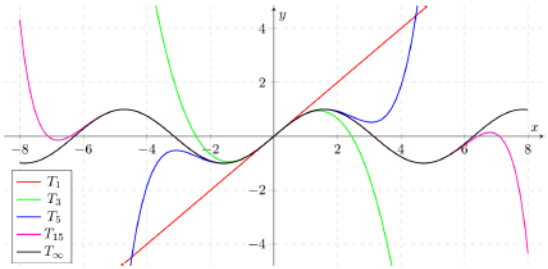
Main Parameters

- n_estimators: Number of estimators (K)
- base_estimator: Weak estimator type
- learning_rate: weights of estimator in final decision (λ)

Boosting example: Gradient Boosting



$$P_k(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots + \frac{f^{(k)}(a)}{k!}(x-a)^k$$



1. Train a weak learner F_0 and compute predictions $\hat{y}_0 = F_0(x)$
2. For $k = 1, \ldots, K$
   - Compute the difference between the target (y) and the prediction of the current learner ($\hat{y}_{k-1}$)
   - Train e weak learner that minimizes the loss function (error):

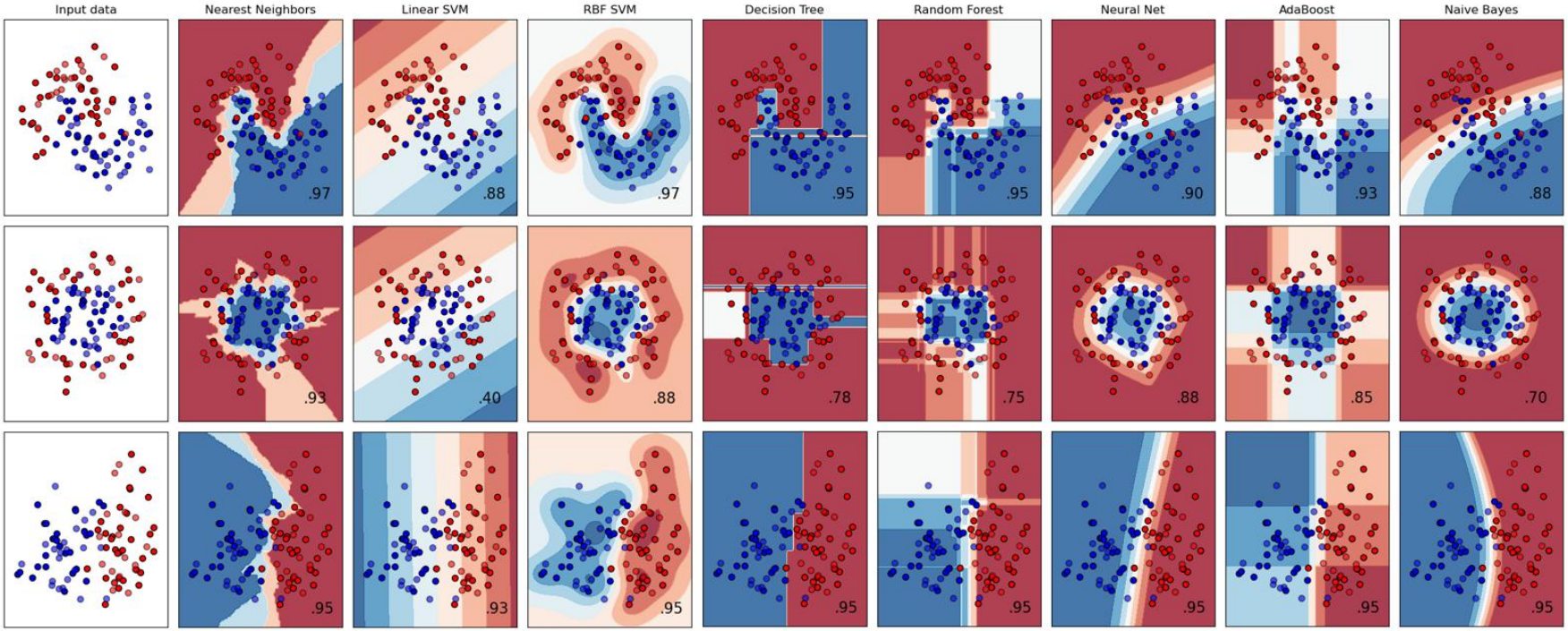
$$f_k = \arg\min_f L_m = \arg\min_f \sum_{i=1}^{n} l(y_i, F_{k-1}(x_1) + f(x_i))$$

   - $F_k = F_{k-1} + \lambda f_k$

Main Parameters

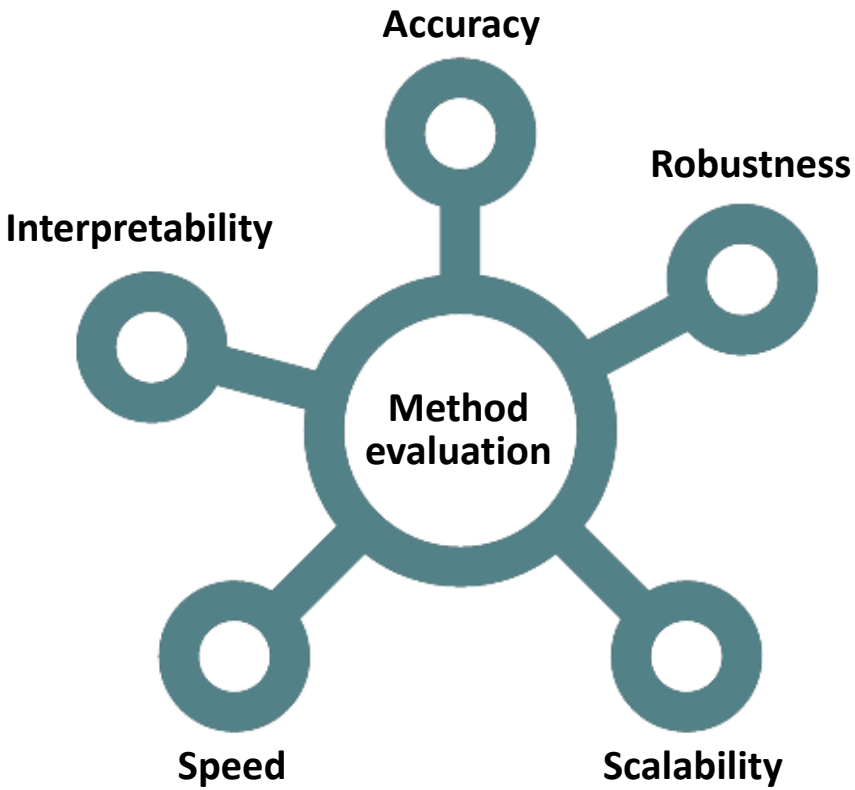
- n_estimators: Number of estimators (K)
- base_estimator: Weak estimator type
- learning_rate: weights of estimator in final decision (λ)

How do classifiers behave?



Source: sklearn, classifier comparison

Requirements

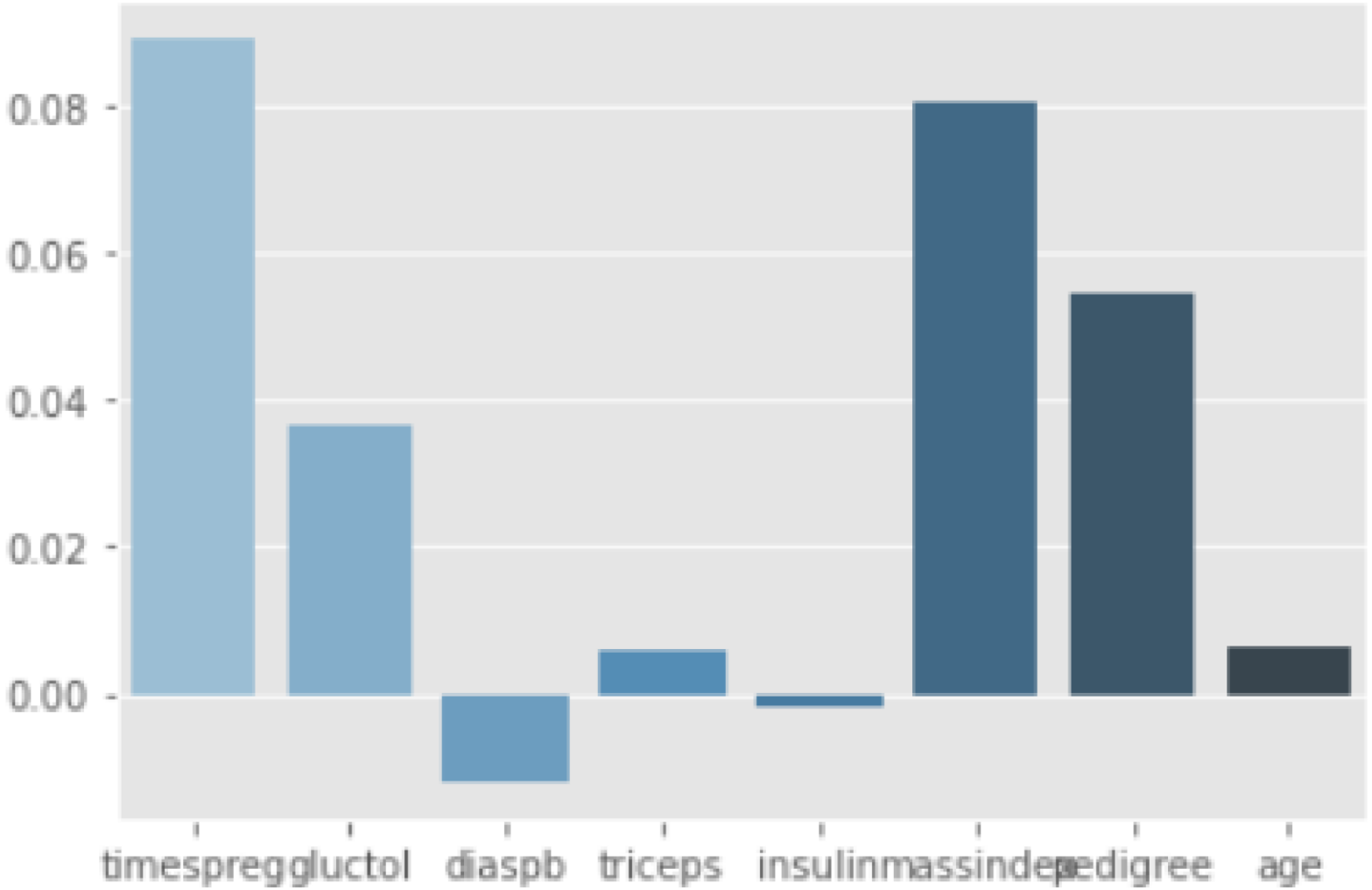


Feature importance

Logistic Regression

Feature importance

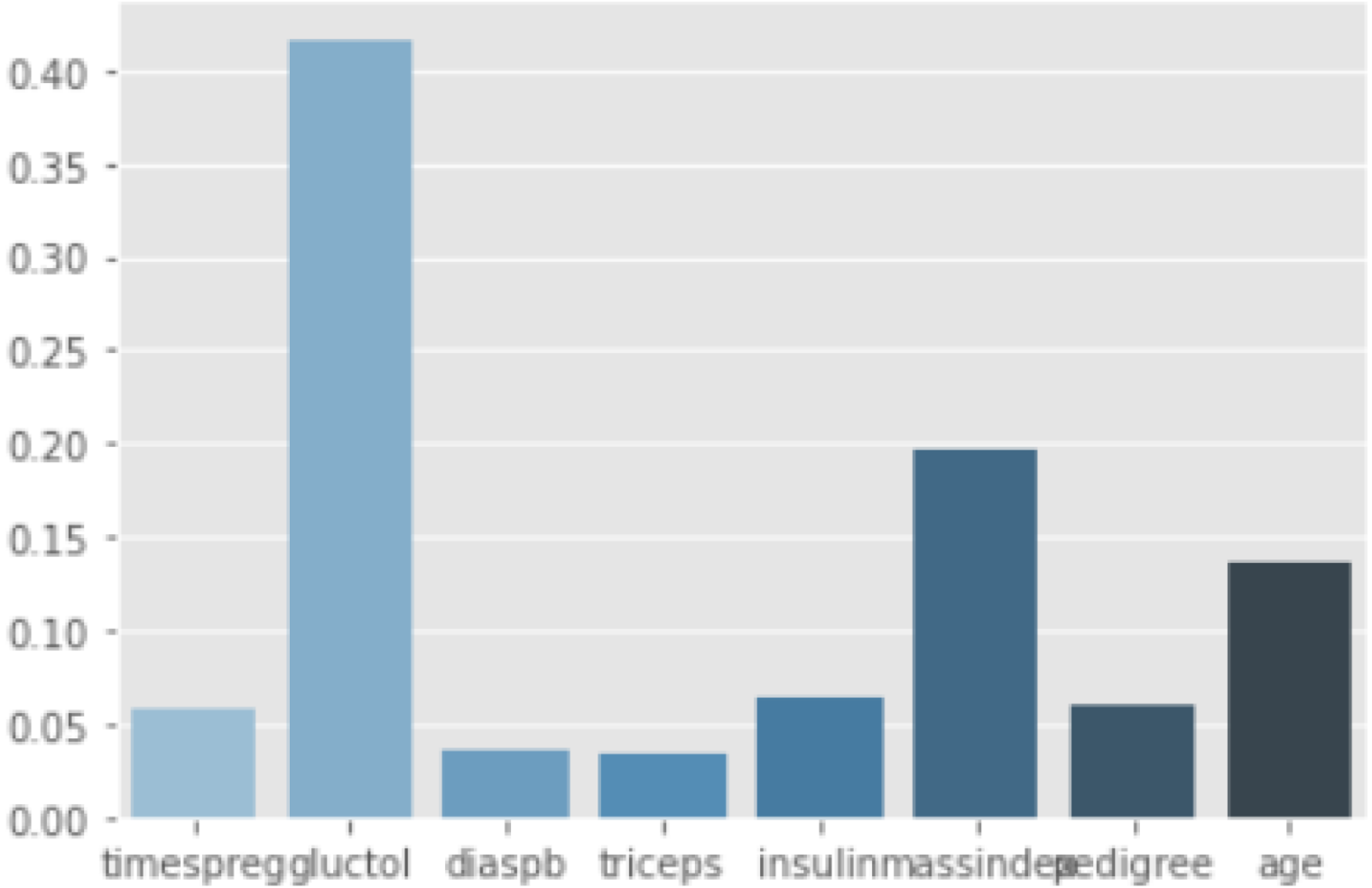
Logistic Regression



Feature importance

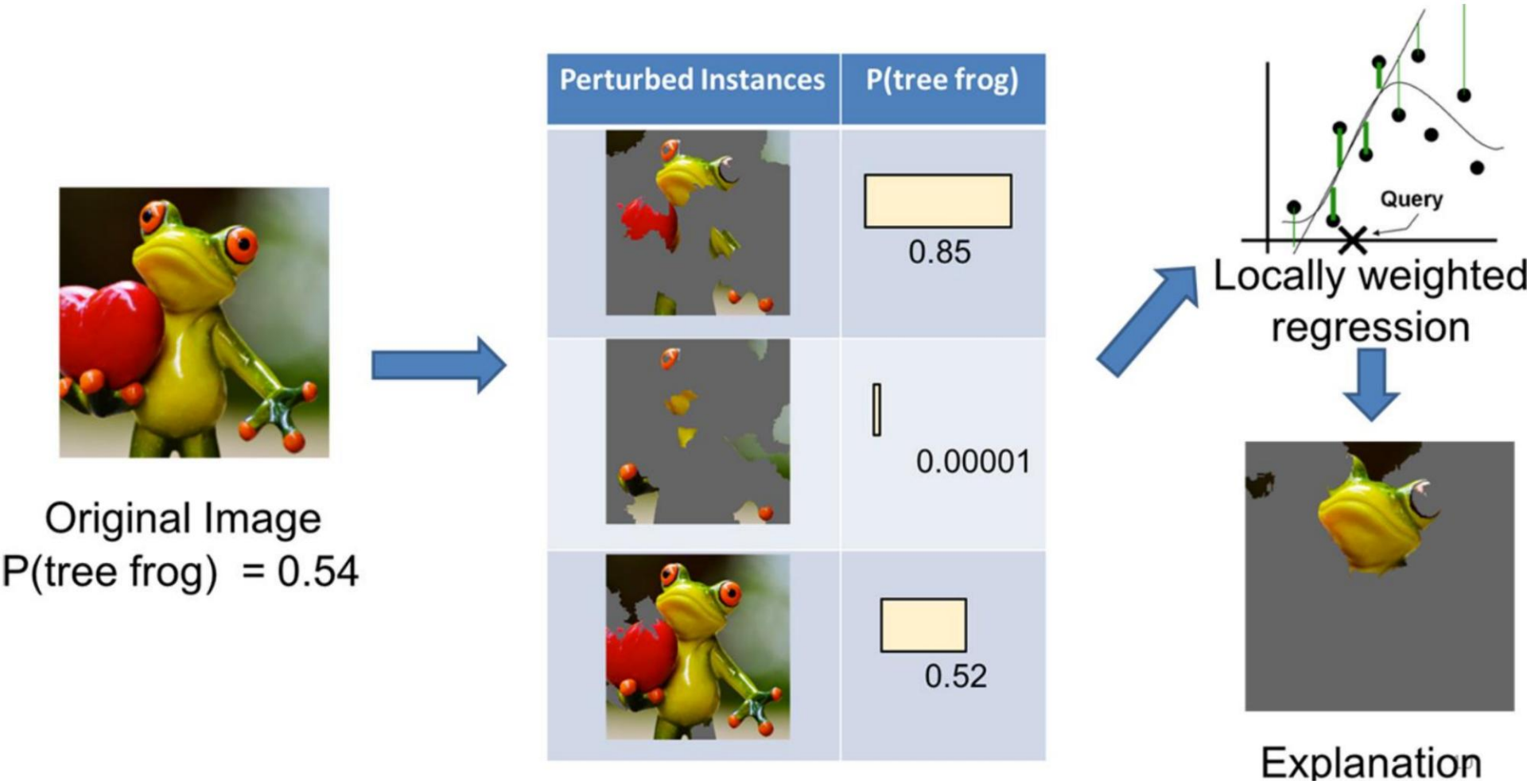
door
machine learning group
POLITECNICO DI MILANO

Random forests (through decision trees)



Feature importance

What if it is not natively supported? Option 1: **LIME**



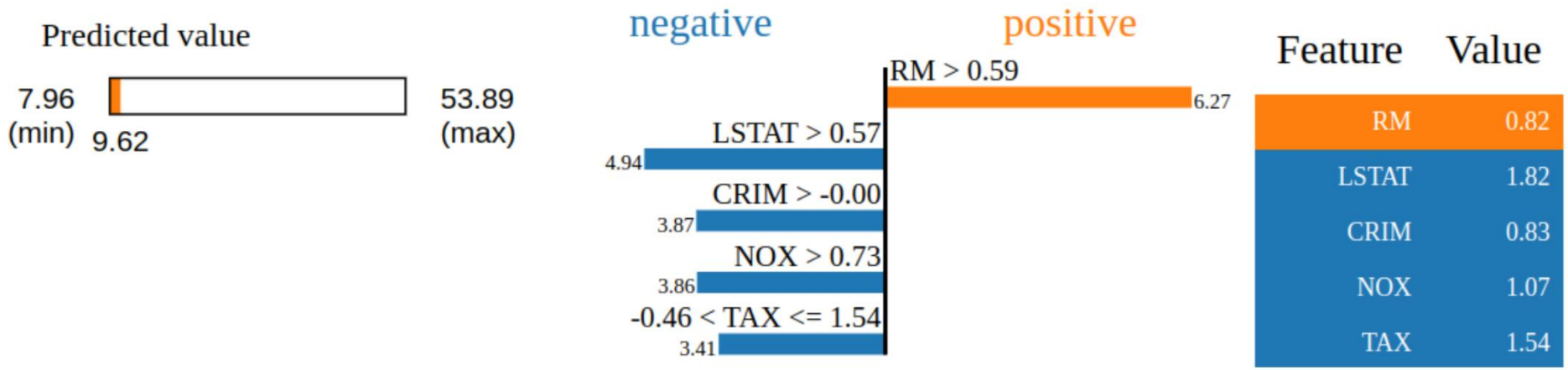
Feature importance

Local Interpretable Model-agnostic Explanations

```
exp_mlp = explainer.explain_instance(X_train[i], mlp.predict, num_features=5)
```

```
Intercept 25.123228322789352
Prediction_local [15.31616231]
Right: 9.621792242553637
```

```
exp_mlp.show_in_notebook(show_table=True)
```



Feature importance

Option 2: **Shapley Values**

$$\phi_j(val) = \sum_{S \subseteq \{x_1,\ldots,x_p\} \setminus \{x_j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \left(val\left(S \cup \{x_j\}\right) - val(S)\right)$$

Feature importance

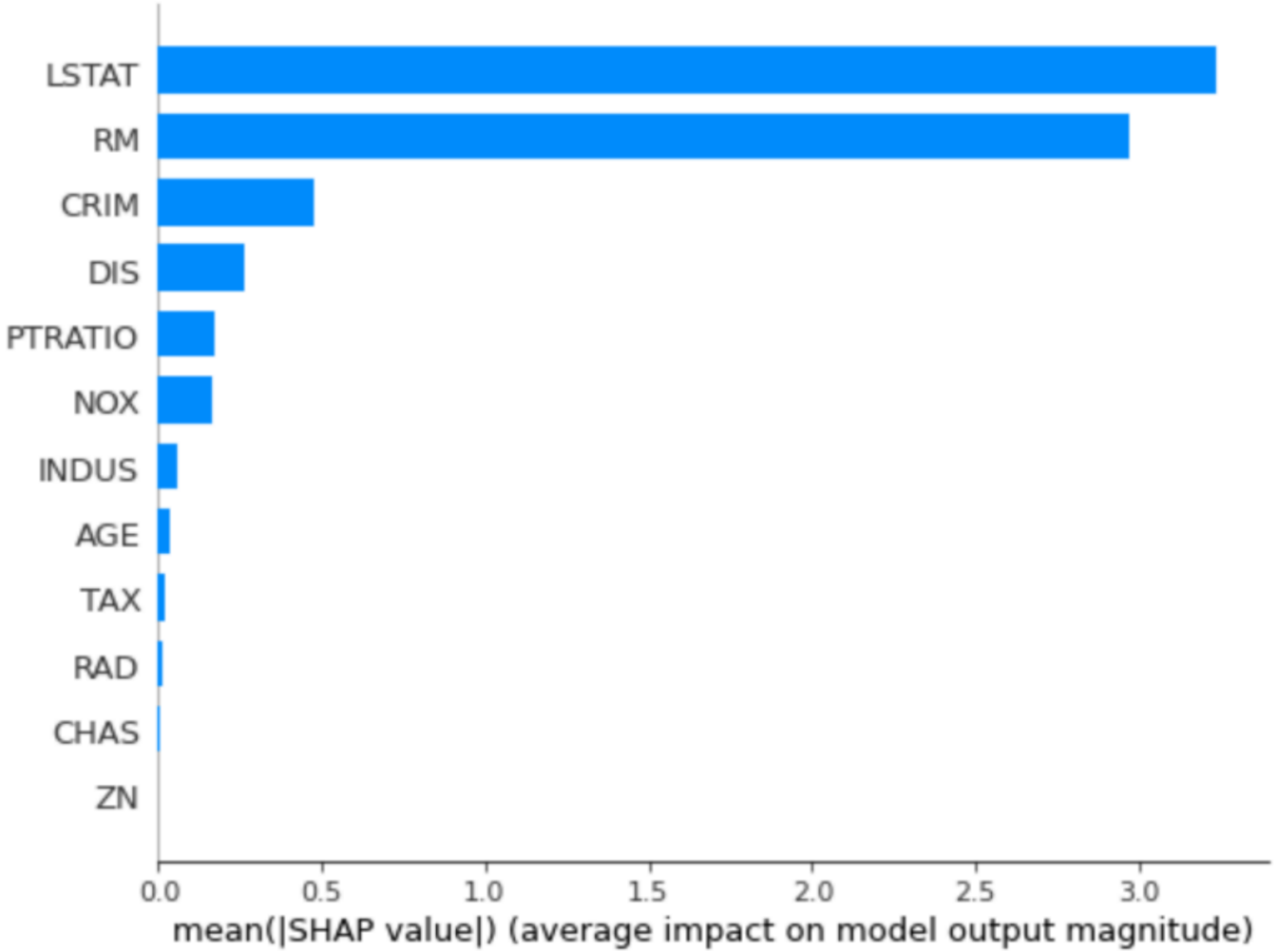
Option 2: **Shapley Values**

$$\phi_j(val) = \sum_{S \subseteq \{x_1,\ldots,x_p\} \setminus \{x_j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \left(val\left(S \cup \{x_j\}\right) - val(S)\right)$$

Feature importance

door
machine learning group
POLITECNICO DI MILANO

Option 2: **Shapley Values**



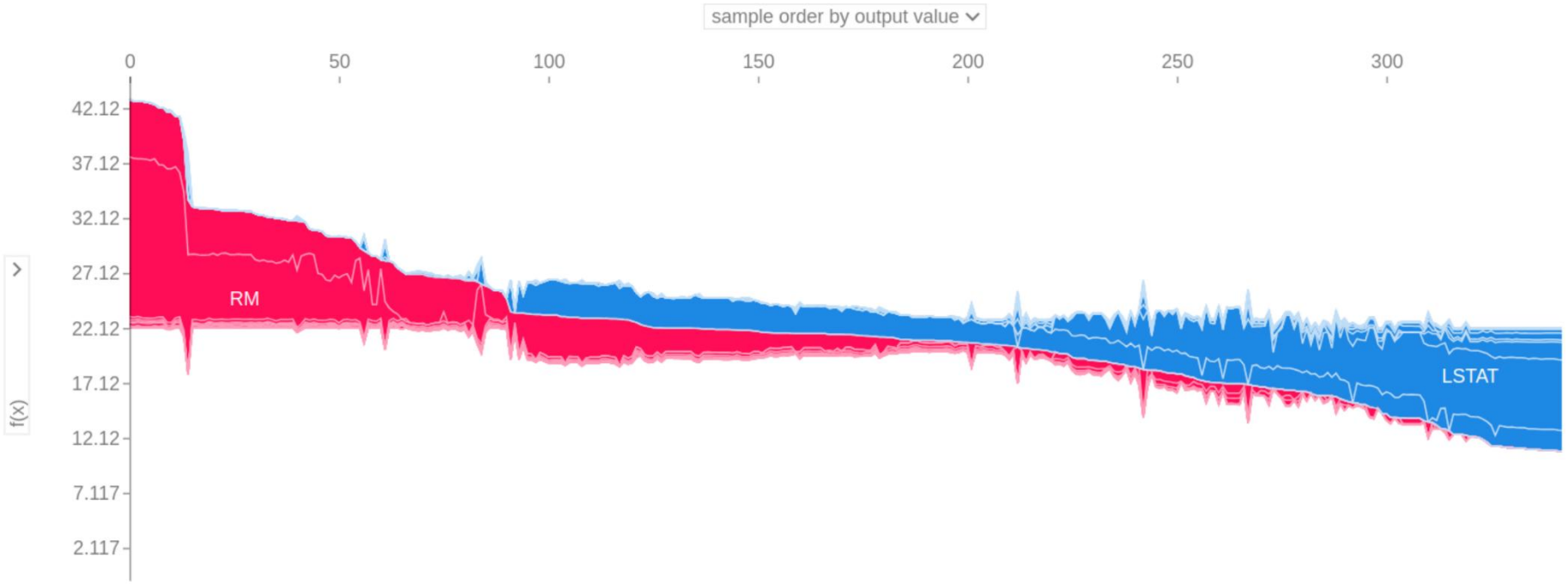
Feature importance

Shapley Values

Feature importance

Shapley Values

```
shap.force_plot(explainer_rf.expected_value,shap_values,features=X_train,feature_names=df_X_train.columns)
```



Feature importance

Shapley Values

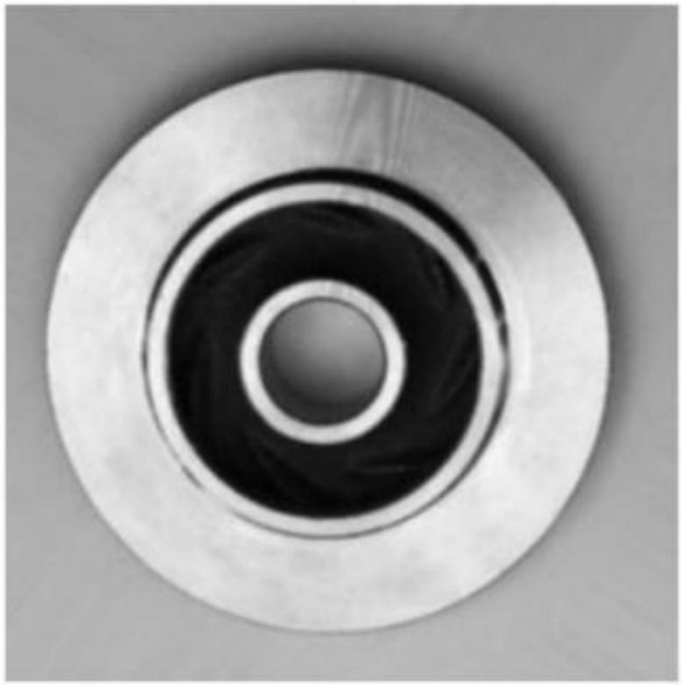
```
i=0

expected_value = shap_values[i,-1]

shap.force_plot(expected_value, shap_values[i,:], df_X_train.iloc[i,:])
```

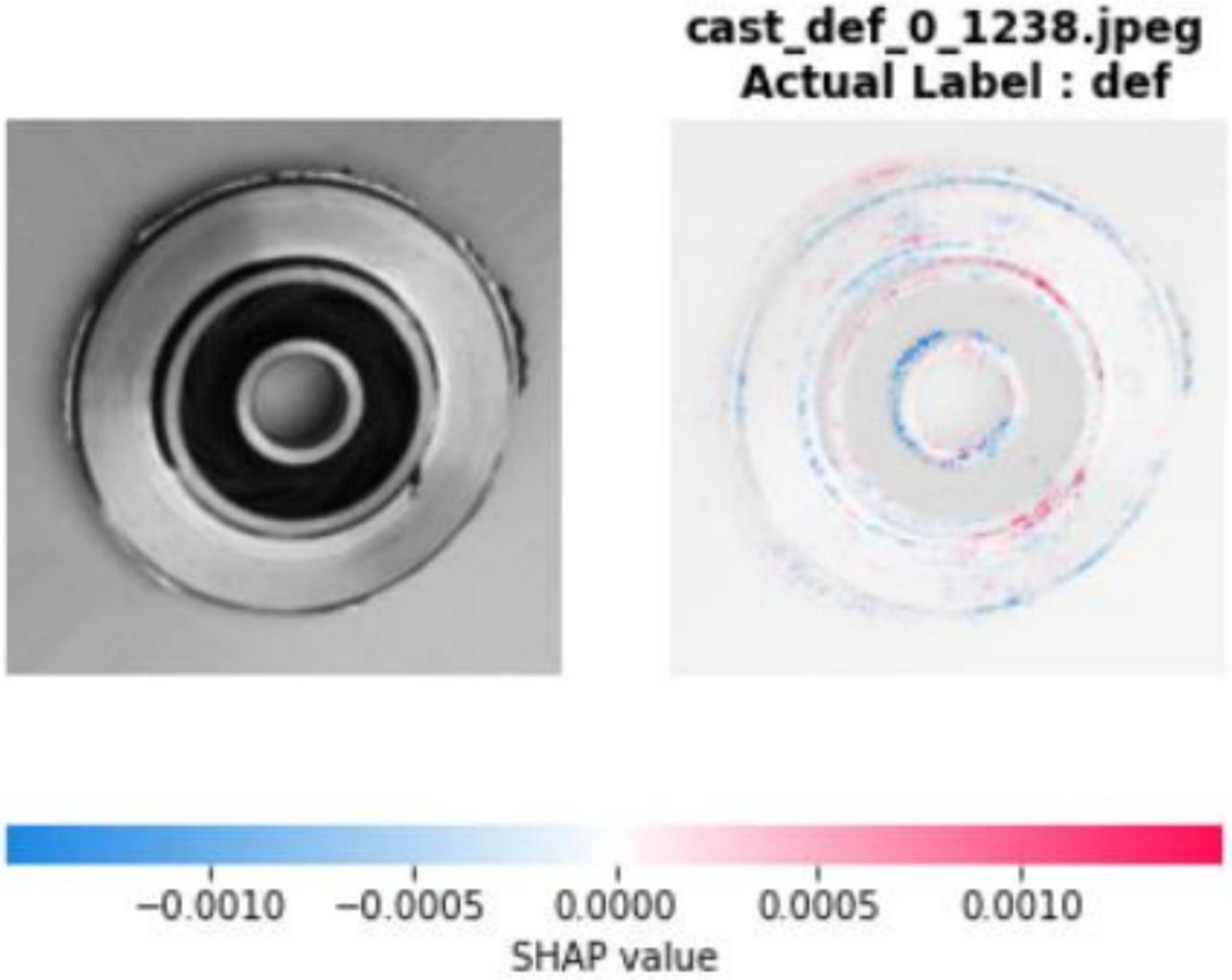
Applications and (overcoming) limitations

ok

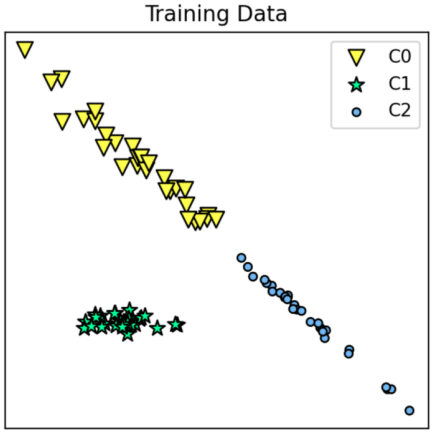


def

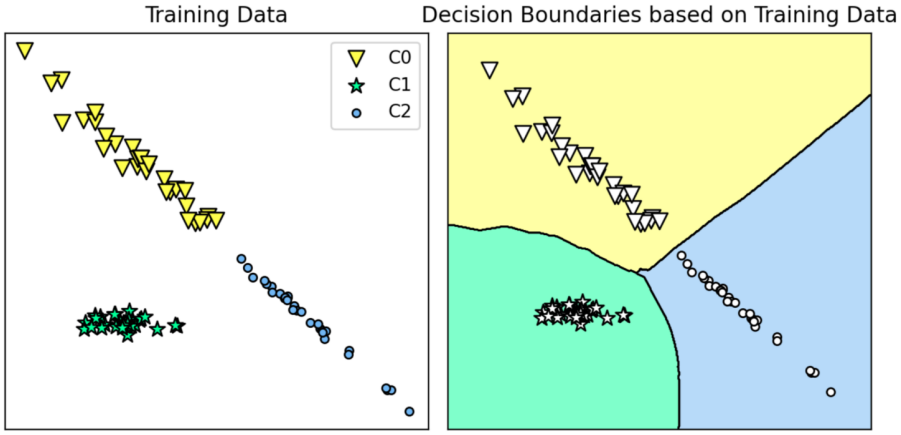
Applications and (overcoming) limitations



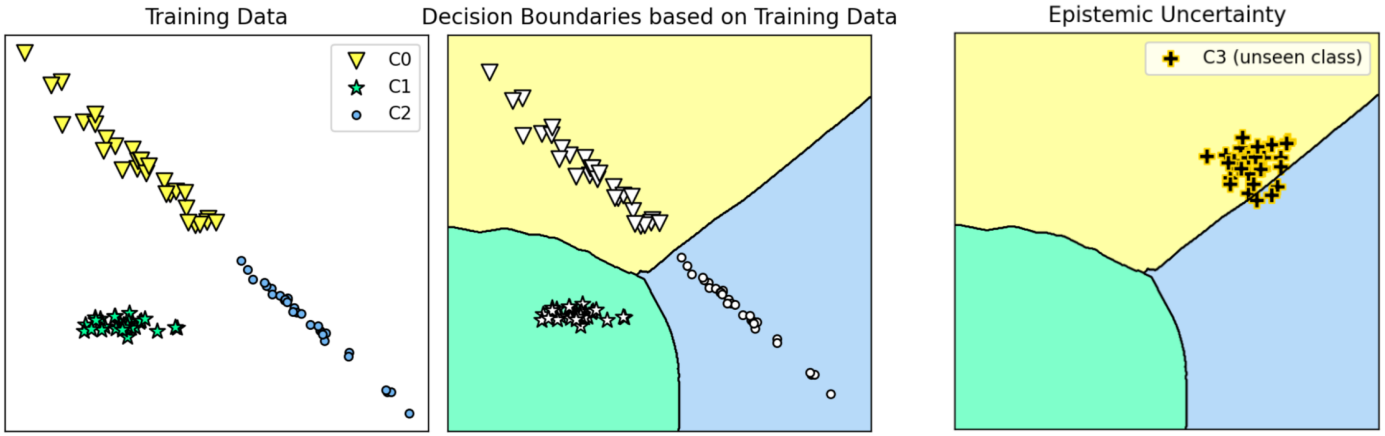
Applications and (overcoming) limitations



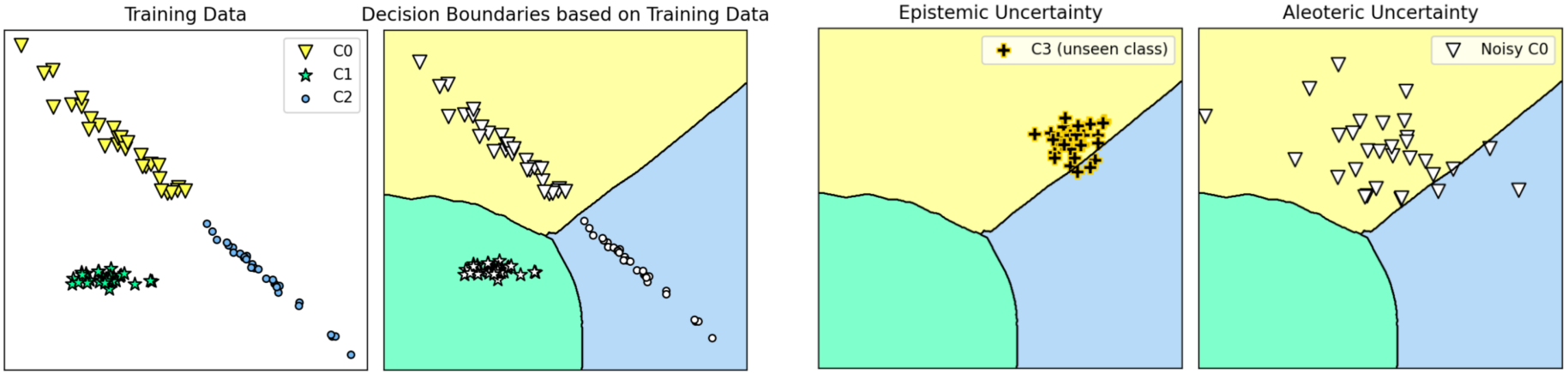
Applications and (overcoming) limitations



Applications and (overcoming) limitations

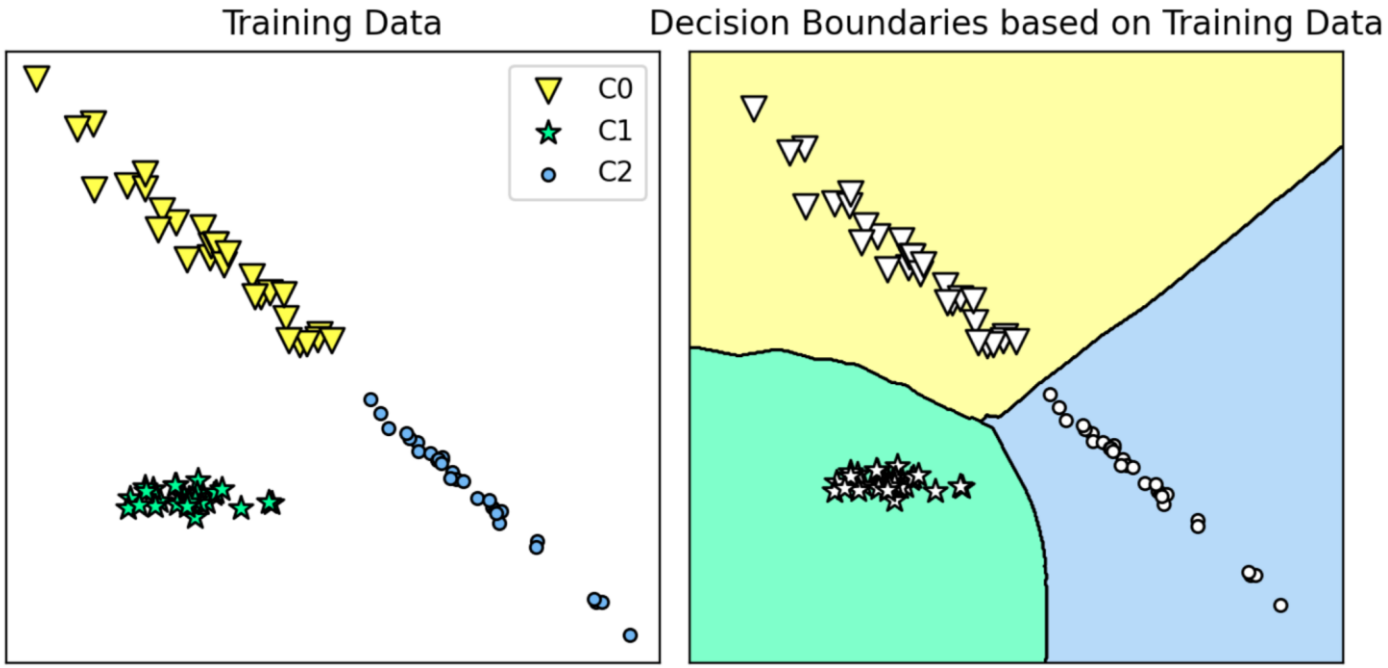


Applications and (overcoming) limitations



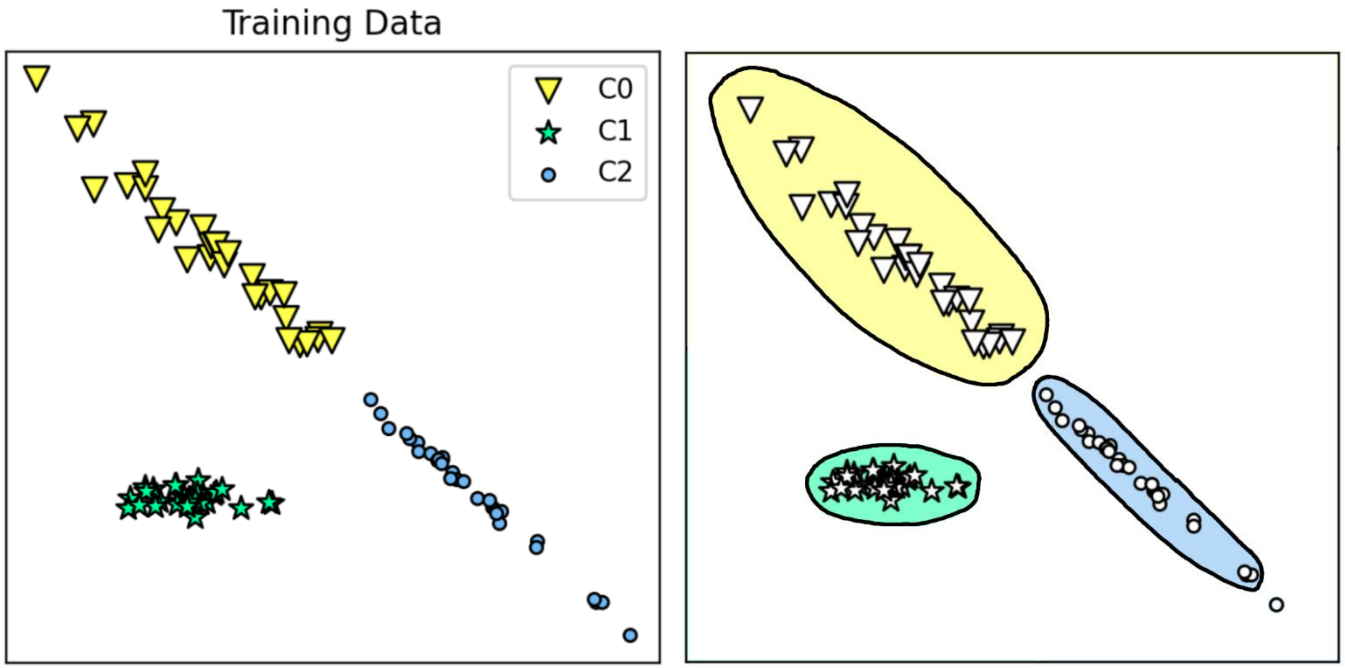
Applications and (overcoming) limitations

Uncertainty-aware classification



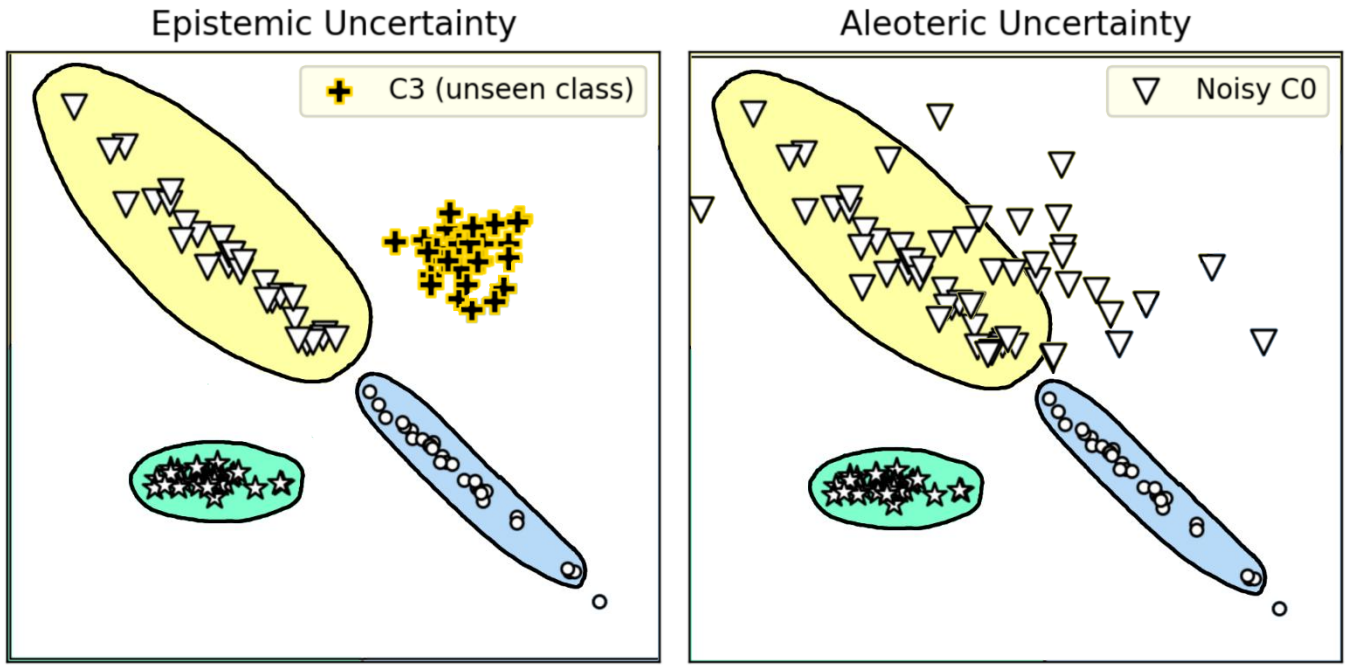
Applications and (overcoming) limitations

Uncertainty-aware classification



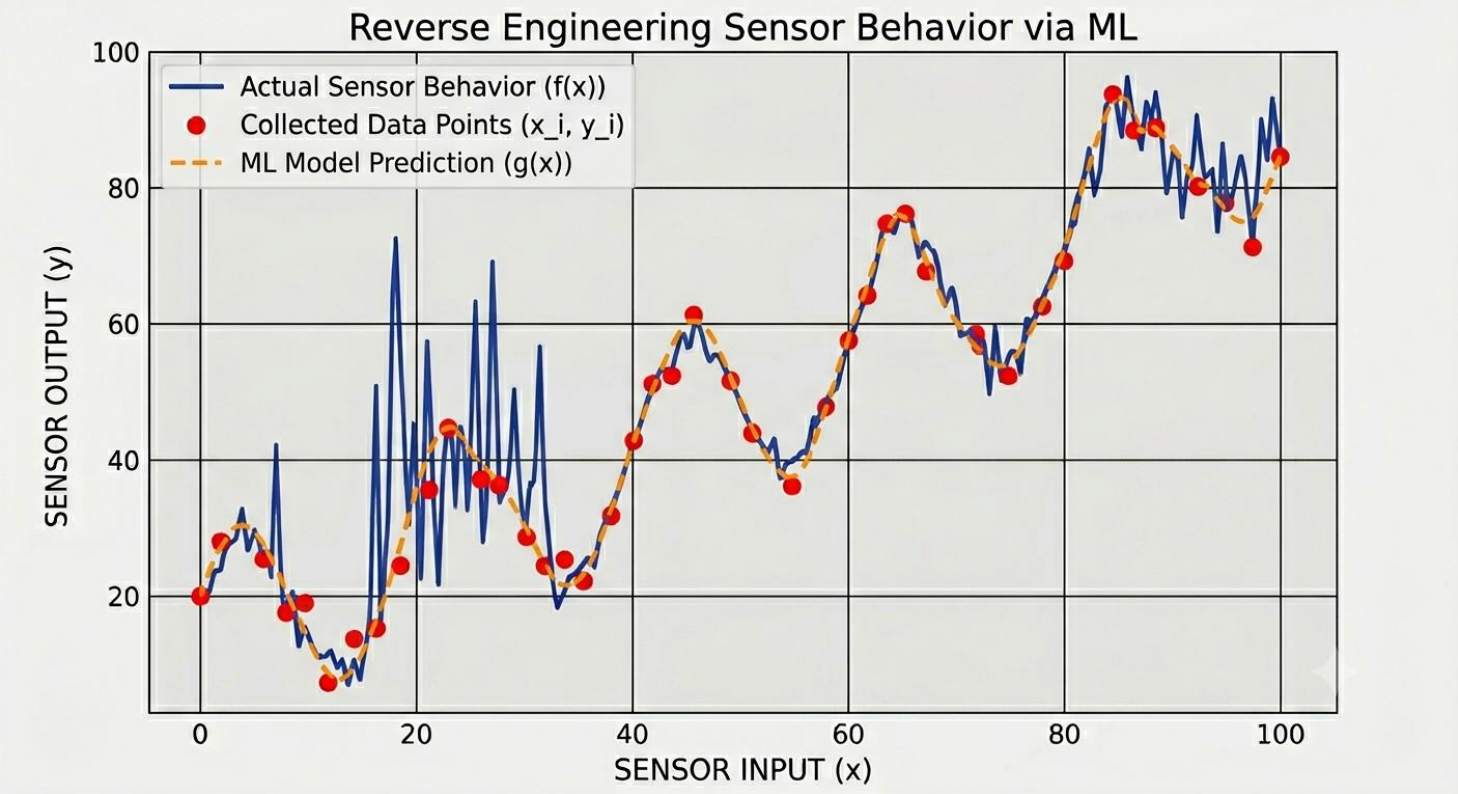
Applications and (overcoming) limitations

Uncertainty-aware classification



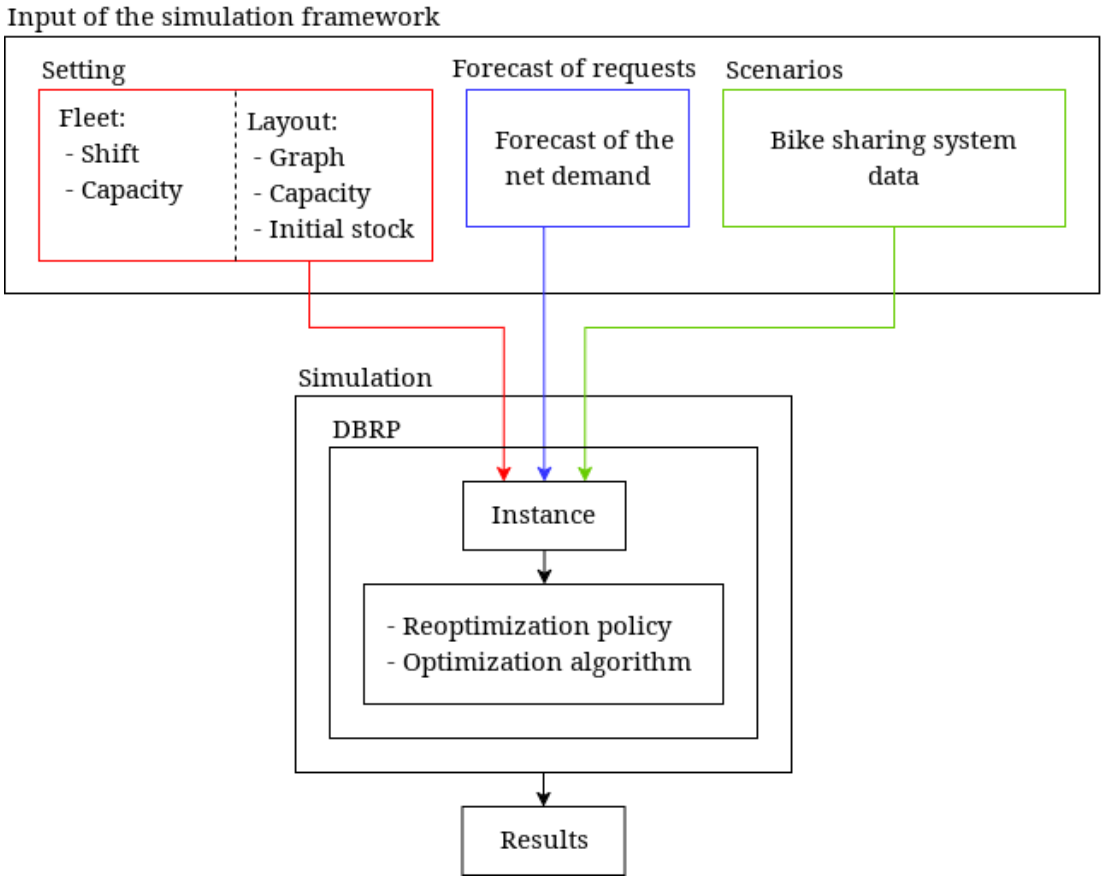
Applications and (overcoming) limitations

Component simulation and reverse engineering



Applications and (overcoming) limitations

Prediction and prescription



Applications and (overcoming) limitations

Prediction and prescription

	Cost	Pred. 1	Pred. 2
c_1	4	5	2
c_2	5	4	7

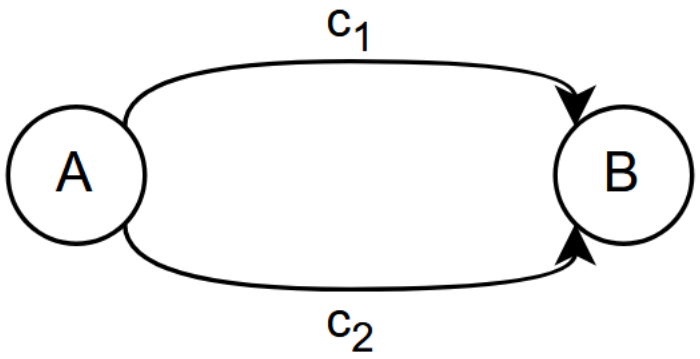
Applications and (overcoming) limitations

Prediction and prescription

	Cost	Pred. 1	Pred. 2
c_1	4	5	2
c_2	5	4	7
MSE	-	1	4

Applications and (overcoming) limitations

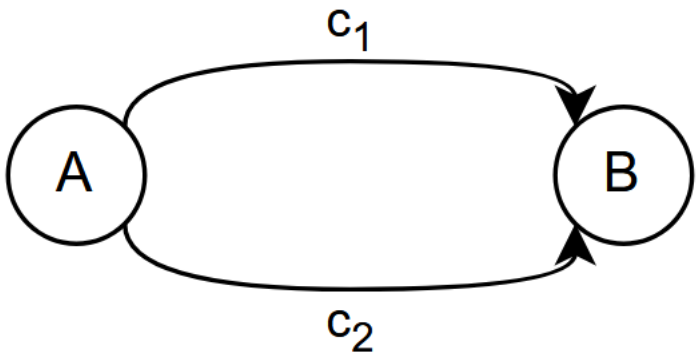
Prediction and prescription



	Cost	Pred. 1	Pred. 2
c_1	4	5	2
c_2	5	4	7
MSE	-	1	4

Applications and (overcoming) limitations

Prediction and prescription



	Cost	Pred. 1	Pred. 2
c_1	4	5	2
c_2	5	4	7
MSE	-	1	4
Opt. gap	-	1	0

Applications and (overcoming) limitations

Prediction and prescription